

UNIVERSIDAD FRANCISCO DE VITORIA

ESCUELA POLITÉCNICA SUPERIOR

COMPUTER SCIENCE DEGREE

FINAL DEGREE PROJECT

RESEARCH TRACK

Word-Level American Sign Language Recognition from Hand Landmarks

Francisco Sandoval Hernández

Tutor: Moisés Martínez Muñóz

CALIFICACIÓN DEL PROYECTO DE FIN DE GRADO

CUALITATIVA:	
NUMÉRICA:	

Conforme Presidente:	**Conforme Secretario:**
Fdo.:	Fdo.:

Conforme Vocal:	**Conforme Vocal:**	**Conforme Vocal:**
Fdo.:	Fdo.:	Fdo.:

Lugar y fecha: Pozuelo de Alarcón, a ____ de _____ de 2026

Acknowledgements

I would like to thank my supervisor, Moisés Martínez Muñóz, for the technical guidance, the steady feedback, and the willingness to push back on every shortcut along the way. The shape of this project owes a great deal to those conversations.

I am also grateful to the Escuela Politécnica Superior of Universidad Francisco de Vitoria for providing the academic environment in which this project took shape, and to the lecturers whose courses laid the foundations on which the rest was built.

Finally, I want to thank my family and friends for their patience and support throughout the long stretches of work that this thesis required.

Abstract

Sign language recognition serves to mitigate communication barriers between deaf or hard-of-hearing signers and non-signing people. This Final Degree Project develops and evaluates a research prototype for word-level American Sign Language (ASL) recognition from short video clips. The work first reviews the accessibility problem, the computer vision methods used for recognition and the technical requirements of the system. It then follows the methodological evolution of the project, from initial approaches based on hand detection and static letter recognition towards a sequence-based model that represents the movement of signs across frames. The final system uses MediaPipe Hands to extract 21 3D hand landmarks per frame, normalises the resulting sequences and classifies each clip with a bidirectional Gated Recurrent Unit (GRU) with a temporal attention mechanism. The experiments evaluate architecture choices, sequence length, cross-dataset evaluation and vocabulary size using the Word-Level American Sign Language (WLASL) and Microsoft American Sign Language (MS-ASL) datasets. The results show consistent performance for reduced vocabularies and across datasets, but accuracy decreases as the number of classes increases. The landmark representation also abstracts away identifying visual information, offering a privacy-conscious alternative to raw video, although the prototype remains a research system rather than a production-ready translator.

Keywords: Sign Language Recognition, Artificial Intelligence, Deep Neural Networks, Computer Vision, Hand Landmarks

Resumen

El reconocimiento de lengua de signos aborda barreras de comunicación entre personas sordas o con discapacidad auditiva y usuarios que no conocen la lengua de signos. Este Proyecto Fin de Grado desarrolla y evalúa un prototipo de investigación para el reconocimiento de palabras en American Sign Language (ASL) a partir de vídeos breves. El trabajo revisa primero el problema de accesibilidad, los métodos de visión por computador utilizados para el reconocimiento y los requisitos técnicos del sistema. A continuación, recoge la evolución metodológica del proyecto, desde los enfoques iniciales basados en detección de manos y reconocimiento estático de letras hacia un modelo basado en secuencias que representa el movimiento de los signos a lo largo de los fotogramas. El sistema final utiliza MediaPipe Hands para extraer 21 puntos clave 3D de la mano por fotograma, normaliza las secuencias resultantes y clasifica cada vídeo mediante una red recurrente con puertas (GRU) bidireccional con un mecanismo de atención temporal. Los experimentos evalúan las decisiones de arquitectura, la longitud de secuencia, la evaluación entre conjuntos de datos y el tamaño del vocabulario utilizando los conjuntos Word-Level American Sign Language (WLASL) y Microsoft American Sign Language (MS-ASL). Los resultados muestran un comportamiento consistente en vocabularios reducidos y entre conjuntos de datos, pero la precisión disminuye al aumentar el número de clases. La representación mediante puntos clave también descarta la mayor parte de la información visual del vídeo original, lo que limita los datos personales que conserva el sistema, aunque el prototipo sigue siendo un sistema de investigación y no un traductor listo para producción.

Palabras clave: reconocimiento de lengua de signos, inteligencia artificial, redes neuronales profundas, visión por computador, puntos clave de la mano

Contents

List of Acronyms **xiii**

1 Introduction **1**
1.1 Problem description . . . 1
1.2 Research scope and approach . . . 1
1.3 Outline . . . 2

2 State of the Art **3**
2.1 From Symbolic to Subsymbolic AI . . . 3
2.2 Deep Neural Networks . . . 4
2.3 Feature Extraction Architectures . . . 5
2.4 Sequence Processing Architectures . . . 8
2.5 Attention Mechanisms . . . 10
2.6 Related Work in Sign Language Recognition . . . 11
2.7 Datasets and Limitations . . . 12

3 Objectives **13**
3.1 Main objective . . . 13
3.2 Specific objectives . . . 13
3.3 Validation methods . . . 14

4 Methodology and Work Plan Overview **16**
4.1 Materials and Resources . . . 16
4.2 Methods . . . 17
4.3 Methodology . . . 18
4.4 Work Plan . . . 20
4.5 Constraints and Limitations . . . 21

5 Technical Development **24**
5.1 System design . . . 24
5.2 Requirements . . . 27
5.3 Datasets . . . 30

6 Experiments **36**
6.1 Experimental Methodology . . . 36
Part I: Quantitative Evaluation . . . 37

6.2 Phase 1: Exploratory Work and Methodological Pivot . . . 37
6.3 Architecture Selection on WLASL30 . . . 38
6.4 Sequence Length Sweep . . . 39
6.5 WLASL30 Validation . . . 41
6.6 Cross-Dataset Validation: MS-ASL50 . . . 41
6.7 Vocabulary Scaling . . . 44
6.8 Limitations and Threats to Validity . . . 45
Part II: Qualitative Evaluation . . . 46
6.9 Qualitative Validation Protocol (In Progress) . . . 46

7 Ethical Implications and Social Impact 50
7.1 AI Based Systems Ethical Impact . . . 50
7.2 Ethical Evaluation Framework: Google's AI Principles . . . 50
7.3 Project Analysis Against Google's AI Principles . . . 51
7.4 Impact Scale of Ethical Aspects . . . 54
7.5 Conclusions of the Ethical Analysis . . . 55

8 My Journey at UFV 57

9 Conclusiones 58
9.1 General Conclusions . . . 58
9.2 Conclusions regarding Project Objectives . . . 58
9.3 Next Steps . . . 58

Bibliography 59

List of Figures

2.1 Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created. 4
2.2 Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [7]. 6
2.3 AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [5]. 6
2.4 MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [8]. 7
2.5 Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created. 8
2.6 Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11]. 9
2.7 Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [13]. 11

4.1 Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration, and final documentation. Source: self-created with GanttProject. 21

5.1 System architecture: data preparation, training and evaluation, and runtime inference tiers. 25

6.1 Row-normalised confusion matrix of the GRU+Attention champion on the WLASL30 test split (34 samples). Source: self-created from experiment outputs. . . 39
6.2 Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs. 40
6.3 Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). The matrices for the other seeds show the same qualitative structure. Source: self-created from experiment outputs. 43

6.4 Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs. 44
6.5 Parallel-coordinates view of the GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green (few classes) to red (many). The widening train–val gap is the hallmark of data starvation. Source: self-created from experiment outputs. 45

List of Tables

3.1 Validation mapping for the specific objectives. 15

5.1 Operational environment for training, landmark extraction, and runtime inference. . . 27
5.2 FR-001: Landmark extraction from video clips 29
5.3 FR-002: Word-level sign classification 29
5.4 FR-003: Reproducible experiment tracking 29
5.5 FR-004: Multi-seed training support 29
5.6 NFR-001: Inference latency 30
5.7 NFR-002: Modular code architecture 30
5.8 NFR-003: Documentation completeness 30
5.9 Comparative overview of all datasets used in this project. 32

6.1 Single-split test results of the five architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the champion is in Section 6.5. 38
6.2 Sequence-length sweep on WLASL30 with 3 seeds per row. Mean ± standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity (≈2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs. 40
6.3 Per-seed test results of the GRU+Attention champion on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs. 41
6.4 Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs. 42
6.5 Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs. 42
6.6 Vocabulary scaling of the GRU+Attention champion on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.4; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests. 44

7.1 Ethical evaluation metrics based on Google's AI Principles [33] 55

List of Acronyms

Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ASL	American Sign Language
AWS	Amazon Web Services
BBox	Bounding Box
BiGRU	Bidirectional Gated Recurrent Unit
BPTT	Backpropagation Through Time
CEC	Constant Error Carousel
CNN	Convolutional Neural Network
CRISP-DM	Cross-Industry Standard Process For Data Mining
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
CV	Cross-Validation
FDP	Final Degree Project
FR	Functional Requirement
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HTTP	Hypertext Transfer Protocol
IoU	Intersection Over Union
JSON	JavaScript Object Notation
JSONL	JavaScript Object Notation Lines
LTS	Long-Term Support
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
MNIST	Modified National Institute Of Standards And Technology

Acronym	Meaning
MS-ASL	Microsoft American Sign Language
NFR	Non-Functional Requirement
OpenCV	Open Source Computer Vision Library
REST	Representational State Transfer
RGB	Red Green Blue
RNN	Recurrent Neural Network
RQ	Research Question
SLR	Sign Language Recognition
SO	Specific Objective
SSDLite	Single Shot Detector Lite
TTA	Test-Time Augmentation
UFV	Universidad Francisco De Vitoria
URL	Uniform Resource Locator
VRAM	Video Random-Access Memory
WLASL	Word-Level American Sign Language
XML	Extensible Markup Language
YAML	Human-Readable Data Serialisation Language
YOLO	You Only Look Once

Chapter 1

Introduction

Sign Language Recognition (SLR) brings together accessibility, computer vision and sequence modelling. Far from being mere gestural shorthand or direct encodings of spoken languages, sign languages are sophisticated natural languages with a unique visual-spatial structure. Meaning is conveyed through a complex interplay of handshape, movement, orientation, location, timing and, in many cases, non-manual cues such as facial expression or body posture. Automatic recognition from video is therefore a demanding problem: the same sign may vary across signers, camera viewpoints, execution speeds, lighting conditions and recording devices.

1.1 Problem description

This project studies American Sign Language (ASL) recognition as an accessibility-oriented research problem. Unlike static image classification, sign recognition is inherently temporal: a single frame may show the handshape and approximate location of a sign, but it cannot represent the direction of movement, the order of articulatory changes or the rhythm of execution. The sign must therefore be interpreted from the ordered relationship between frames.

The task addressed in this work is isolated word recognition. The system receives a short video clip and must assign it to a word gloss, that is, a written label used to annotate a sign. This task is more limited than continuous sign language translation, but it preserves the temporal nature of signs and provides a suitable scope for a research-oriented undergraduate project.

1.2 Research scope and approach

The central research question of this work is whether compact visual representations can provide enough information for temporal models to classify short ASL video clips into word glosses. The project evaluates different neural architectures under reproducible experimental conditions, with the objective of identifying a configuration that balances accuracy and computational cost. The work does not aim to replace professional interpreters or to deliver a production-ready translator.

The experimental work is grounded on public ASL datasets, mainly WLASL [1] and MS-ASL [2]. A mobile and server-based runtime prototype is also developed as a proof of concept, although the

main scientific contribution is the experimental evaluation of the recognition approach rather than the deployment of an assistive product.

1.3 Outline

The remainder of the work is organised as follows:

- Chapter 2 reviews the State of the Art, providing the theoretical foundation in computer vision and deep learning required for this work. It examines convolutional neural networks and pose estimation, specifically MediaPipe Hands, before addressing temporal modelling through recurrent networks, Long Short-Term Memory (LSTM) layers, Gated Recurrent Unit (GRU) layers, Transformers and attention mechanisms. The chapter also evaluates previous research in SLR to establish the context for the current approach.
- Chapter 3 defines the main objective, the specific objectives, and the validation methods used to determine whether each objective has been achieved.
- Chapter 4 outlines the methodology and project work plan. It details the iterative, experiment-driven approach adopted throughout development and delineates the main project phases. Finally, it addresses the technical resources and constraints that influenced the scope and design of the project.
- Chapter 5 presents the technical development and system architecture. It provides a rigorous account of the data preprocessing pipeline and the experimental framework established for training and evaluation. The chapter concludes with a description of a functional proof of concept, demonstrating a real-time implementation through a mobile client and a FastAPI-based inference server [3].
- Chapter 6 reports the experimental evaluation. It includes an architecture comparison and sequence-length analysis, followed by validation on the WLASL30 and MS-ASL50 datasets. The chapter also addresses cross-dataset validation, vocabulary scaling and the qualitative validation design, and concludes with a critical discussion of the main limitations observed during experimentation.
- Chapter 7 analyses the ethical implications and social impact of the project, structured around the Google Artificial Intelligence Principles and ethical framework. It provides a critical examination of accessibility, privacy and algorithmic bias, while addressing the requirements for responsible deployment and the importance of reproducibility. Finally, it evaluates the broader social value of assistive technologies through these ethical guidelines.
- Chapter 8 reflects on the academic journey at Universidad Francisco de Vitoria and relates the project to the competences developed during the degree.
- Chapter 9 presents the conclusions and future work, summarising the main findings, the limitations of the current prototype, and possible directions for extending the research.

Chapter 2

State of the Art

This chapter reviews the technical foundations that support this project. It begins with the move from symbolic Artificial Intelligence (AI) to data-driven learning, then introduces Deep Neural Networks (DNNs), feature extraction architectures, sequence processing layers, attention mechanisms, related Sign Language Recognition (SLR) work and the datasets most relevant to American Sign Language (ASL) recognition.

2.1 From Symbolic to Subsymbolic AI

Early AI systems were symbolic, representing knowledge through explicit logic and predefined rule sets. They solved problems by applying human-authored rules, often organised as conditional decision trees. While this paradigm flourished within constrained environments, it proved inadequate for perception tasks, such as computer vision and speech, due to the inherent noise, ambiguity, and high-dimensionality of real-world data. This phenomenon, termed the "knowledge acquisition bottleneck", implies that as task complexity scales, the maintenance of handcrafted rule bases becomes untenable.

In the context of sign language, symbolic approaches could encode basic gesture prototypes, but they failed to capture the nuanced articulatory variation found across signers, camera angles and execution speeds. A fixed rule for a hand shape may describe an idealised sign, yet the observed motion in video changes with signer anatomy, partial occlusion, lighting and the temporal rhythm of the execution. These sources of variation make exhaustive symbolic description impractical.

Subsymbolic AI addresses the same class of problems through statistical learning. Instead of hard-coding expert rules for every possible case, a learning system estimates parameters from examples. This transition was driven by advances in neural networks and statistical learning theory, culminating in deep learning methods that learn hierarchical features directly from raw inputs [4]. By treating perception as a high-dimensional optimisation problem, these systems reduce the noise and ambiguity that previously led to the knowledge acquisition bottleneck.

In the SLR domain, this transition mirrors the evolution of the field: moving away from attempts to define a perfect hand-shape rules and towards architectures that extract latent features from raw video data. Consequently, the burden of feature engineering shifts from the human expert to the learning

algorithm, allowing for a more fluid and robust interpretation of manual dynamics that aligns with the success seen in modern speech recognition and object detection frameworks.

2.2 Deep Neural Networks

A neural network is a parameterised function composed of layers. Each layer receives an input vector, applies a linear transformation and passes the result through a non-linear activation function. A DNN stacks several such layers, allowing the model to compose simple transformations into progressively more abstract representations. Figure 2.1 shows this structure as a mapping from input features to an output prediction through hidden layers.

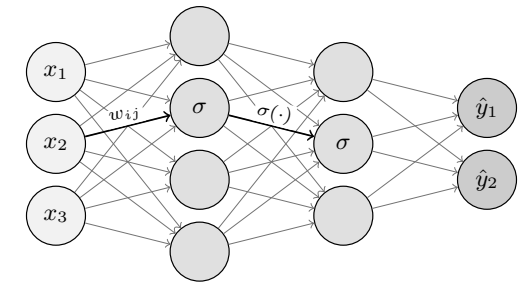


Figure 2.1: Basic DNN structure. Inputs are transformed through hidden layers into an output prediction. Source: self-created.

For a layer l, the computation can be written as:

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right) \tag{2.1}$$

where $\mathbf{W}^{(l)}$ contains the learned weights, $\mathbf{b}^{(l)}$ is the bias vector and σ is the activation function. The Rectified Linear Unit (ReLU), defined as $\text{ReLU}(x) = \max(0, x)$, is a common activation because it introduces non-linearity while keeping the computation simple. The output layer depends on the task: classification layers estimate class probabilities, whereas regression layers estimate continuous quantities such as coordinates.

Training adjusts the parameters $\theta = \{\mathbf{W}, \mathbf{b}\}$ so that the function f_θ maps inputs to the desired targets. The model computes a prediction, compares it with the ground truth through a loss function and updates the parameters by gradient-based optimisation. In this sense, a DNN is not a list of explicit rules but an optimised function approximator.

This formulation explains the link between DNNs and feature extraction. Intermediate layers do not only transform the data numerically; they also produce representations that make the target task easier. In computer vision, these representations may correspond to edges, textures, object parts or pose coordinates. Feature extraction therefore denotes the process of converting raw observations into informative variables that a later classifier or sequence model can use.

2.3 Feature Extraction Architectures

Feature extraction architectures convert video frames into representations that preserve relevant visual information while reducing irrelevant variation. In SLR, two families are especially important. Pixel-based techniques operate directly on image tensors, where each pixel stores colour or intensity values. Pose estimation techniques first detect anatomical points and then represent the signer through coordinates. The distinction matters because it determines whether the model learns from appearance information or from a geometric abstraction of the body and hands.

2.3.1 Pixel-Based Techniques

A digital image can be represented as a tensor $\mathbf{x} \in \mathbb{R}^{H \times W \times C}$, where H and W are height and width and C is the number of colour channels. Pixel-based techniques use this tensor as the direct input to the model. This preserves all visual information, including hand texture, background, illumination and camera artefacts.

Convolutional Neural Networks (CNNs) are the standard DNN architecture for pixel-based computer vision [4]. A convolutional layer applies a small matrix of learned parameters, called a kernel or filter, across local regions of the image. At each spatial position, the kernel computes a response that indicates whether a visual pattern is present. The collection of responses produced by one kernel is a feature map.

For input tensor $\mathbf{x}$ and output feature map $\mathbf{y}$, a convolutional layer can be written as:

$$\mathbf{y}_{i,j,k} = \sigma\left(\sum_{c=1}^{C}\sum_{u,v}\mathbf{w}_{u,v,c,k}\mathbf{x}_{i+u,j+v,c} + b_k\right) \tag{2.2}$$

where $\mathbf{w}$ is the set of kernels, b_k is the bias for feature map k and σ is the activation function. Because the same kernel is reused across the image, convolutional layers encode translation equivariance: if a pattern shifts in the image, its corresponding activation shifts in the feature map.

CNN architectures stack convolutional, activation and pooling layers. Pooling reduces the spatial resolution of feature maps by summarising local neighbourhoods, commonly through maximum or average values. This supports local translation invariance, since small shifts in the input produce similar pooled activations. Early layers usually respond to oriented edges and simple textures; deeper layers combine those responses into larger shapes and object-level cues, as illustrated in Figure 2.2. AlexNet [5] and ResNet [6] are established examples of this hierarchical design.

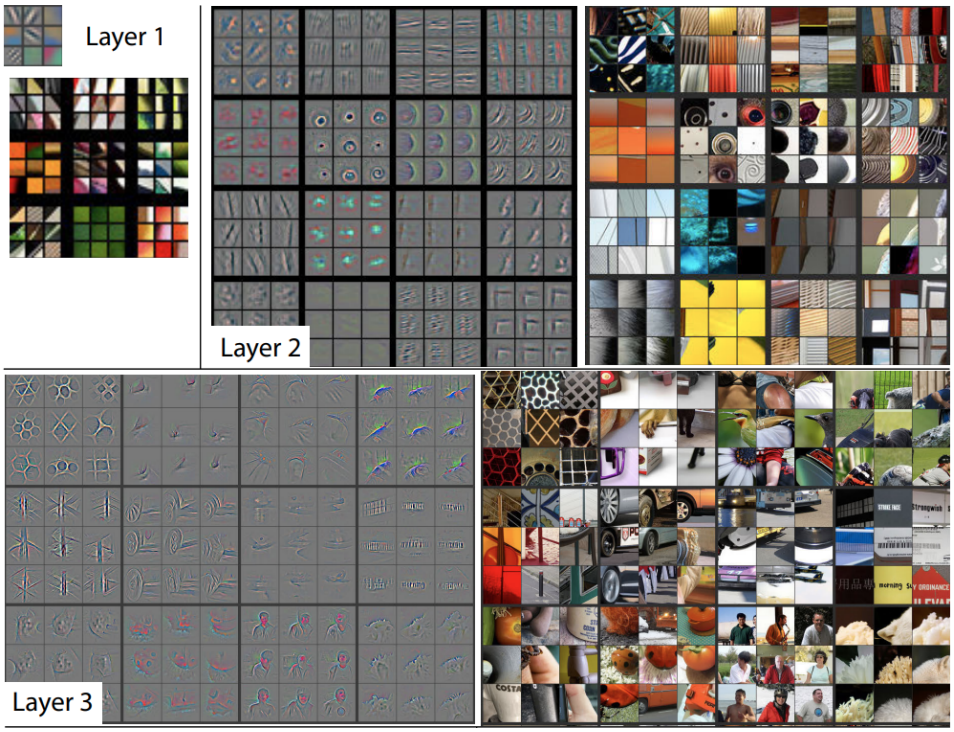


Figure 2.2: Hierarchical feature representations learned by a CNN. Layer 1 responds to oriented edges and simple colour gradients; Layer 2 captures corners, circular patterns and basic textures; Layer 3 represents more complex textures and object parts. Adapted from Zeiler and Fergus [7].

Figure 2.3 shows the AlexNet architecture. The diagram is relevant here because it illustrates the repeated pattern of convolution, pooling and fully connected layers that made deep pixel-based recognition practical.

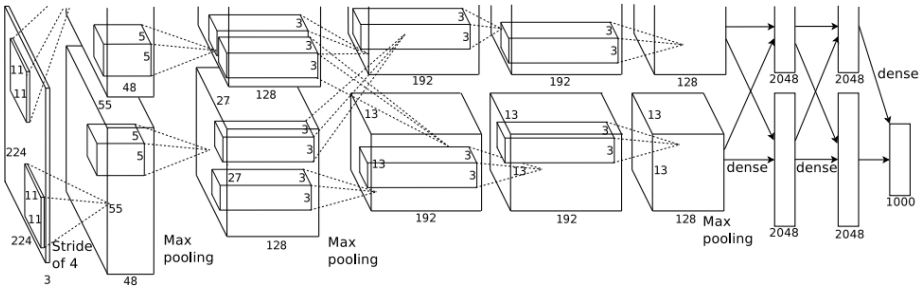


Figure 2.3: AlexNet architecture, showing stacked convolutional and pooling layers followed by fully connected layers. Source: Krizhevsky et al. [5].

In SLR, pixel-based CNNs can learn discriminative visual patterns directly from frames. Their main

limitation is that they also receive information that may not define the sign, including background, skin tone and recording conditions. When the available dataset is limited, this can encourage the model to fit appearance correlations instead of sign structure.

2.3.2 Pose Estimation Techniques

Pose estimation is the task of estimating the spatial location of anatomical points from an image or video frame. In machine learning, regression refers to predicting continuous numerical values rather than selecting one class from a fixed set. A landmark is a tracked point with image or world coordinates. In hand pose estimation, many landmarks correspond to joints, which are anatomical articulations such as knuckles, while fingertip landmarks mark the end of each finger. A regression-based pose estimator therefore predicts landmark coordinates rather than a discrete class label.

Pose pipelines are commonly organised as top-down or bottom-up systems. A top-down pipeline first detects the relevant object or body part, then estimates landmarks inside the detected region. A bottom-up pipeline first predicts candidate landmarks across the image, then groups them into instances. Both approaches convert raw image evidence into a structured geometric representation.

MediaPipe Hands [8] is a widely used hand pose estimation system. It uses the top-down structure shown in Figure 2.4: a hand detector proposes a region of interest, and a landmark model estimates hand coordinates inside that cropped region.

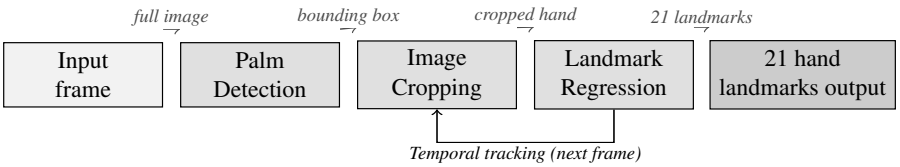


Figure 2.4: MediaPipe Hands pipeline. The palm detector locates the hand region on the first frame or when tracking is lost; the landmark model then regresses 21 three-dimensional landmarks inside the cropped region. Subsequent frames reuse the landmarks from the previous frame, avoiding running palm detection on every frame. Source: self-created, adapted from Zhang et al. [8].

The process consists of the following stages:

1. **Palm detection. Input:** full image frame. **Operation:** a single-shot detector estimates a bounding region around the palm. **Output:** a rectangle defining the hand region of interest.
2. **Image cropping. Input:** original frame and detected rectangle. **Operation:** the image is cropped and aligned around the hand. **Output:** a normalised hand image for the landmark model.
3. **Landmark regression. Input:** cropped hand image. **Operation:** a DNN regresses the coordinates of 21 hand landmarks. **Output:** a structured set of three-dimensional landmark coordinates.
4. **Temporal tracking. Input:** landmarks and region of interest from the previous frame. **Operation:** the pipeline reuses recent location information when the hand remains visible. **Output:** a frame-to-frame landmark sequence.

For SLR, landmark sequences provide a compact description of hand shape and movement. They remove much of the raw appearance signal and make it possible to derive additional geometric features such as bone vectors, joint angles and distances between fingers. The cost of this abstraction is that pose estimators may fail under occlusion, motion blur or unusual camera angles, and hand-only landmarks omit non-manual cues such as facial expression and upper-body movement.

2.4 Sequence Processing Architectures

Isolated sign recognition is a temporal classification problem. The input is not a single image but an ordered sequence of feature vectors, one per frame or sampled time step. A sequence processing layer therefore receives $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T\}$ and produces a representation that accounts for temporal order.

2.4.1 Recurrent Layers

Recurrent Neural Networks (RNNs) process a sequence by maintaining a hidden state. At time t, the layer combines the current input $\mathbf{x}_t$ with the previous hidden state $\mathbf{h}_{t-1}$:

$$\mathbf{h}_t = \phi(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{2.3}$$

where ϕ is an activation function. The same parameters are shared at every time step, as shown in Figure 2.5. This sharing lets the layer process sequences of different lengths while applying the same transition rule throughout the video.

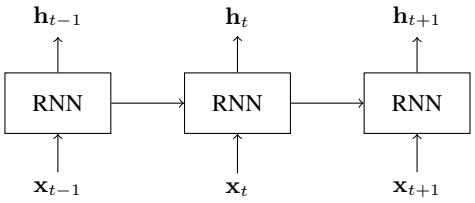


Figure 2.5: Unrolled recurrent layer. The same recurrent computation is applied at each time step while the hidden state carries information forward. Source: self-created.

Training recurrent layers uses Backpropagation Through Time (BPTT), which unfolds the recurrent computation across the sequence and propagates the loss backwards through the unfolded graph. As the number of time steps increases, standard recurrent layers frequently encounter the vanishing gradient problem [9]. The gradient signal becomes too small to preserve information from early time steps, which limits long-range temporal modelling.

The recurrent layer above is the simple recurrent case. The following advanced layer types are described in the next subsections:

- Long Short-Term Memory (LSTM) layers, which introduce a cell state and gates to regulate information flow.
- Gated Recurrent Unit (GRU) layers, which simplify the LSTM design while retaining gated memory.

- Transformer layers, which process all positions through self-attention rather than recurrence.

2.4.2 Long Short-Term Memory Layers

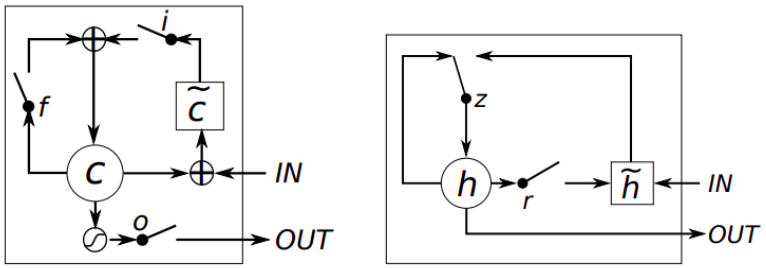
Long Short-Term Memory (LSTM) layers were proposed by Hochreiter and Schmidhuber [10] to improve the modelling of long temporal dependencies. An LSTM layer maintains two vectors: the hidden state $\mathbf{h}_t$, which is exposed to the next layer or output, and the cell state $\mathbf{C}_t$, which carries memory along the sequence.

The LSTM cell regulates this memory through gates. The forget gate controls which information is removed from the cell state, the input gate controls which new information is written and the output gate controls which part of the cell state becomes the hidden state. Figure 2.6 (a) represents these gates as f, i and o, and shows how the cell state C provides a direct path for information flow.

This gated memory path is the mechanism referred to as the Constant Error Carousel. It reduces the decay of the error signal during BPTT and allows the layer to retain information over longer time spans than a simple recurrent layer. The cost is computational: each time step requires several linear transformations, increasing both parameter count and training time.

2.4.3 Gated Recurrent Unit Layers

Gated Recurrent Unit (GRU) layers were introduced by Chung et al. [11] as a simpler gated recurrent alternative. A GRU layer removes the separate cell state and stores memory directly in the hidden state $\mathbf{h}_t$. It uses two gates, shown in Figure 2.6(b): the update gate z and the reset gate r.



(a) Long Short-Term Memory (b) Gated Recurrent Unit

Figure 2.6: Comparison of gated recurrent units. (a) LSTM cell with input, forget and output gates. (b) GRU cell with update and reset gates. Source: Chung et al. [11].

The GRU gate equations at time t are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z) \tag{2.4}$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r) \tag{2.5}$$

The reset gate controls how much previous information is used to compute the candidate hidden state $\tilde{\mathbf{h}}_t$. The update gate controls the interpolation between the previous hidden state and the candidate state:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{2.6}$$

When $\mathbf{z}_t$ is close to zero, the layer copies most of the previous hidden state. When it is close to one, the layer replaces it with new information. This gives GRU layers a compact mechanism for modelling temporal dependencies with fewer parameters than LSTM layers.

2.4.4 Transformer Layers

Transformer layers replace recurrence with self-attention [12], an extension of the attention mechanism described in Section 2.5. Instead of processing the sequence step by step, a Transformer computes interactions between all positions in parallel. Each input position is projected into a query, a key and a value vector. The similarity between queries and keys determines how much each position attends to the others, and the values are combined according to those attention weights.

This design supports parallel computation across the sequence, which is useful for large datasets and long inputs. However, Transformers do not include the same sequential inductive bias as recurrent layers. They require positional information to represent order, and their performance in small SLR datasets may depend strongly on the amount of data available for training. For this reason, Transformer layers are relevant to the comparison of sequence models, but they do not replace recurrent layers by default.

2.5 Attention Mechanisms

Attention mechanisms assign different weights to different parts of an input sequence. In neural machine translation, Bahdanau et al. [13] introduced attention to avoid compressing a full source sentence into a single fixed-length vector. The same idea applies to video recognition: not every frame contributes equally to the final prediction.

Given encoder states $\{\mathbf{h}_t\}_{t=1}^{T}$, an attention module computes a score e_t for each time step and normalises the scores with a softmax function:

$$\alpha_t = \frac{\exp(e_t)}{\sum_{k=1}^{T} \exp(e_k)}, \quad e_t = a(\mathbf{s}, \mathbf{h}_t) \tag{2.7}$$

where $a(\cdot)$ is a scoring function and α_t is the attention weight assigned to time step t. The context vector is then computed as:

$$\mathbf{c} = \sum_{t=1}^{T} \alpha_t \mathbf{h}_t \tag{2.8}$$

Figure 2.7 shows this weighted combination: the decoder state attends to encoder annotations through weights $\alpha_{t,1}, \alpha_{t,2}, \ldots, \alpha_{t,T}$.

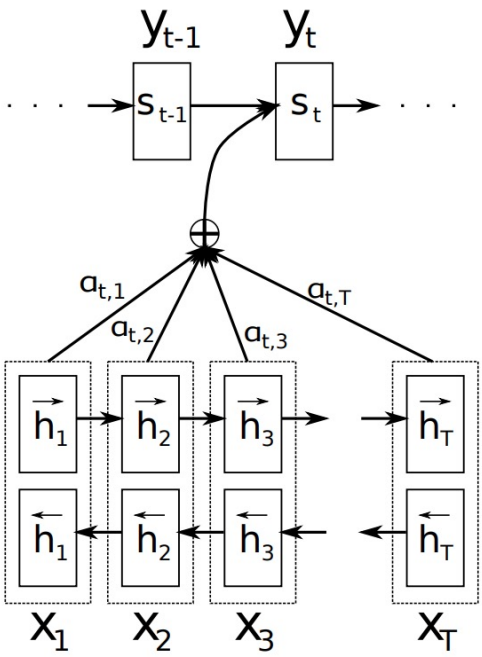


Figure 2.7: Attention mechanism in an encoder-decoder architecture. The context vector is a weighted sum of encoder annotations. Source: Bahdanau et al. [13].

For example, in an SLR clip of the sign *drink*, the first frames may show the hand moving towards the mouth, the central frames may contain the class-defining motion and the final frames may show the hand returning to rest. Temporal attention can assign higher weights to the central frames and lower weights to the preparation and recovery frames. This does not remove the need for a sequence model; it adds a learned weighting stage over the temporal representation produced by that model.

2.6 Related Work in Sign Language Recognition

SLR research combines object detection, feature extraction and temporal classification. Earlier vision pipelines often treated hands as regions to be detected before classification. Modern object detectors such as Faster Region-based Convolutional Neural Network (Faster R-CNN) [14], Single Shot MultiBox Detector (SSD) [15], EfficientDet [16] and the You Only Look Once (YOLO) family [17], [18] frame this problem as locating objects and assigning class scores in images. YOLO is especially relevant for real-time hand detection because it performs detection in a single forward pass, and YOLOv8 is a current implementation frequently used in practical detection pipelines [19].

Survey work distinguishes between isolated sign recognition, continuous sign recognition and sign language translation [20], [21], [22]. Isolated recognition assigns a single gloss to a trimmed clip, which is the scope of this project. Continuous recognition must segment and classify signs from longer streams, while translation must produce spoken-language text and account for grammar, context and non-manual markers.

Word-level ASL recognition is strongly associated with WLASL and MS-ASL. Li et al. [1] introduced WLASL together with benchmark comparisons for word-level recognition from video, while Joze and Koller [2] introduced MS-ASL as a larger ASL benchmark. Both works helped move isolated SLR evaluation away from small curated vocabularies and towards larger, noisier video collections.

Architecturally, the literature has moved from handcrafted descriptors towards DNN-based pipelines. Pixel-based CNNs learn frame-level visual features, recurrent layers model temporal evolution and attention modules weight the most informative time steps. Transformer-based work such as Sign Language Transformers [23] and STMC-Transformer [24] shows how attention-based sequence models have been applied to continuous recognition and translation settings.

Pose-based methods replace raw frames with landmark sequences, reducing dimensionality and limiting the influence of background appearance. Bohacek and Hruz [25] proposed a pose-based Transformer for word-level SLR, while Sign2Pose [26] combines pose features and Transformer-based gloss prediction on WLASL. Large-scale work such as BSL-1K [27] also highlights that dataset scale, weak supervision and cross-dataset transfer remain central concerns in sign recognition research. This project is situated in the pose-based line: MediaPipe Hands provides hand landmarks, and the recognition model evaluates recurrent and attention-based temporal processing.

2.7 Datasets and Limitations

SLR systems depend strongly on the datasets available for training and evaluation. Unlike general image recognition, sign language recognition still lacks abundant large-scale benchmarks across languages, domains and signer populations. Dataset design therefore shapes what a recognition model can learn: the vocabulary determines the classification space, recording conditions affect visual variation and annotation practice defines the gloss labels used for evaluation.

WLASL [1] and MS-ASL [2] are the main ASL references for word-level video recognition. They provide public benchmarks at a scale suitable for comparing temporal models, but they also reflect common limitations in SLR research. Many clips originate from online video sources, which introduces variable camera quality, inconsistent clip duration and dataset attrition when source videos are removed or made private. Class imbalance also remains a recurring issue, since frequent signs have many more examples than rare signs.

Auxiliary resources such as static alphabet datasets or hand-detection corpora are useful for engineering stages such as detector training or letter-level experiments. They should not be treated as word-level SLR benchmarks because they do not evaluate temporal sign dynamics. For that reason, this chapter treats them as secondary resources and leaves the dataset composition and preparation details to the technical development chapter.

Chapter 3

Objectives

This chapter defines the purpose of the project, decomposes it into specific and verifiable objectives, and states how each objective is validated in the remainder of the thesis. The objectives describe the research and project outcomes pursued by the work. They are therefore different from the software requirements presented in Section 5.2, which specify the functional and non-functional behaviour expected from the implemented system.

3.1 Main objective

The main objective of this Final Degree Project is to design, implement, and experimentally validate a research prototype for word-level American Sign Language (ASL) recognition from short video clips, using hand-landmark representations and temporal neural models. The purpose is not to build a production-ready translator, but to study whether a compact geometric representation of hand motion can support feasible and reproducible isolated-word sign recognition.

The project also has an accessibility motivation. By exploring automatic recognition of ASL glosses, the work studies how computer vision and deep learning techniques can contribute to assistive communication technologies. The technical system is therefore a means to evaluate a recognition approach, its limitations, and its ethical implications, rather than an end product intended for direct deployment.

3.2 Specific objectives

The main objective is decomposed into six specific objectives. Each one is defined as a measurable project outcome and is later mapped to a validation method in Section 3.3.

- **SO1: Analyse the technical and scientific background of sign language recognition.** This objective covers the relevant foundations of computer vision, pose estimation, MediaPipe Hands, sequence modelling, recurrent neural networks, GRUs, and attention mechanisms, with emphasis on their applicability to word-level sign language recognition.
- **SO2: Define a landmark-based data representation for ASL video clips.** This objective focuses on transforming raw ASL video clips into normalised sequences of three-dimensional

hand landmarks suitable for temporal modelling, while documenting the datasets, preprocessing decisions, and limitations introduced by landmark extraction.

- **SO3: Design and compare candidate temporal models for word-level ASL recognition.** This objective evaluates alternative temporal architectures under a controlled WLASL30 benchmark and identifies the model most suitable for the available data regime.
- **SO4: Validate the selected recognition pipeline through reproducible experimental protocols.** This objective evaluates the selected model using standard classification metrics and protocols that reduce dependence on a single random seed or split, including multi-seed evaluation and stratified cross-validation.
- **SO5: Assess the generalisation and scalability of the recognition pipeline.** This objective tests whether the selected approach transfers to an independent ASL dataset and analyses how performance changes as the WLASL vocabulary size increases.
- **SO6: Contextualise the technical contribution through a runtime proof of concept and ethical analysis.** This objective integrates the recognition pipeline into an end-to-end mobile/server prototype as a proof of concept and analyses the accessibility, privacy, bias, reproducibility, and responsible-deployment implications of the approach.

3.3 Validation methods

Table 3.1 maps each specific objective to the method used to validate it and to the thesis evidence where the committee can verify the result. Detailed numerical results are intentionally left to the Experiments chapter; this section only defines how objective achievement is assessed.

Table 3.1: Validation mapping for the specific objectives.

Objective	Validation method	Evidence in the thesis
SO1	The objective is validated by presenting a structured review of the main techniques needed to understand the proposed system, including convolutional models, pose estimation, MediaPipe Hands, recurrent models, GRUs, and attention mechanisms.	State of the Art chapter (Chapter 2) and the cited literature used to justify the methodological choices.
SO2	The objective is validated by documenting the pipeline that converts ASL videos into normalised landmark sequences and by describing the datasets, extraction process, preprocessing steps, and data-quality constraints.	System design and dataset description in Section 5.1 and Section 5.3; methodological pivot in Section 6.2.
SO3	The objective is validated by comparing candidate temporal architectures on the same WLASL30 benchmark using Top-1, Top-5, Macro-F1, and Balanced Accuracy, and by selecting the strongest model under the controlled comparison.	Architecture selection in Section 6.3 and Table 6.1.
SO4	The objective is validated through the multi-seed protocol, the sequence-length sweep, stratified 5-Fold Cross-Validation, and the reporting of standard metrics across repeated runs.	Experimental methodology in Section 6.1; sequence-length sweep in Table 6.2; WLASL30 validation in Section 6.5 and Table 6.3.
SO5	The objective is validated by retraining and evaluating the same pipeline on MS-ASL50 without architectural changes and by measuring performance degradation across progressively larger WLASL vocabularies.	Cross-dataset validation in Section 6.6, Table 6.4, and Table 6.5; vocabulary scaling in Section 6.7 and Table 6.6.
SO6	The objective is validated by describing the runtime prototype architecture, defining a qualitative validation protocol for live use, and evaluating the ethical and social implications of the system as a research prototype.	Runtime architecture in Section 5.1; qualitative protocol in Section 6.9; ethical and social-impact analysis in Chapter 7.

Chapter 4

Methodology and Work Plan Overview

This chapter describes how the project was actually carried out: which materials and resources were used, which technical methods are applied where, what process governed the day-to-day decisions, in what sequence the work was done, and which constraints shaped every choice. The substance of *what was built* is reserved for the Technical Development chapter (Section 5.1 and following), and the substance of *what was discovered experimentally* is reserved for the Experiments chapter (Section 6.1 and following); this chapter explains the *way of working* that connects them.

4.1 Materials and Resources

The project required four classes of resources: hardware, software stack, datasets, and supervision. Each class is described briefly below and cross-referenced to the chapters where the technical detail lives.

Hardware. All training, evaluation, and landmark extraction were performed on a single local workstation whose specification is the operational environment of Section 5.1.2; the full table of components and version constraints is Table 5.1 and is not repeated here. The relevant element for the work-plan discussion is the GPU: an NVIDIA RTX 3050 Ti Laptop with 4 GB of VRAM. The decision to keep the entire training pipeline on a local GPU rather than on a cloud platform such as Google Colab or AWS was deliberate. Local execution gives bit-for-bit reproducibility between consecutive runs, costs nothing per run (which mattered because the iterative methodology described in Section 4.3 relaunches experiments often), and removes the network-dependent latency that would otherwise penalise short interactive runs. The 4 GB VRAM ceiling is, in turn, the dominant hardware constraint discussed in Section 4.5.

Software stack. The training and evaluation pipeline runs on Python 3.10 with PyTorch 2.x and CUDA 12.x. Pose estimation uses MediaPipe Hands 0.10+, video decoding uses OpenCV 4.8+, and version control is provided by Git with GitHub as remote host. The mobile client is built with

React Native and Expo; the inference server uses FastAPI 0.111 with Uvicorn 0.29. Documentation is authored in LaTeX. The exact version constraints are locked in `pyproject.toml` for the training package and in `04_server/requirements.txt` for the inference server, so that the operational environment is reproducible from a clean clone of the repository (cross-reference: Section 5.1.2 and the reproduction guide at `docs/reproducibility_guide.md`).

Datasets. Six datasets are used across the project: EgoHands and the Kaggle Hand Dataset for hand localisation, Sign Language MNIST and the ASL Alphabet for letter classification, and WLASL and MS-ASL for word recognition. Each dataset is formally defined in Section 5.3, where attribute schema, licence considerations, and the role of each dataset in the experimental pipeline are described. They are listed here only to make the resource inventory complete; the work-plan discussion treats the datasets as external inputs whose acquisition, audit, and preparation costs are accounted for in the corresponding phases of Section 4.4.

Supervision and consultation resources. The project was supervised by Moisés Martínez Muñóz through scheduled tutor meetings held weekly or biweekly depending on the phase of the work. The most consequential technical decisions of the project such as the pivot from pixel-based letter recognition to landmark-based word recognition, the adoption of stratified K-fold cross-validation in place of test-time augmentation, the cross-dataset extension to MS-ASL50 in addition to WLASL, and the introduction of the multi-seed protocol, did not originate as solo decisions but as outcomes of these meetings. The archived meeting notes preserve the directives that triggered each shift, and an internal audit document was used to trace the meeting references cited in this chapter. Consultation resources beyond the tutor consist of the literature reviewed in chapter 2 (notably the WLASL benchmark, MediaPipe Hands, and the Bahdanau attention formulation) and a small number of open-source repositories used as reference implementations: the WLASL benchmark distribution and the MediaPipe Hands toolkit.

4.2 Methods

This section is a compact catalogue of the technical methods the project relies on, presented without re-justification. The theoretical background of each method is the subject of chapter 2, and the empirical evidence for each one is the subject of the Experiments chapter (Section 6.1 and following); the role of this section is to make the inventory explicit so that the rest of the chapter can refer to it.

The perception frontend uses MediaPipe Hands, a top-down two-stage pipeline that first detects the palm and then regresses 21 three-dimensional keypoints per detected hand. The choice to operate on landmarks instead of raw pixels is the central methodological decision of the project and is justified in detail in Section 6.2; the relevant point here is that all downstream methods consume a landmark vector of 63 features per frame.

The temporal model is a bidirectional Gated Recurrent Unit with 256 hidden units per direction and two stacked layers, followed by a Temporal Attention head implemented as an additive (Bahdanau-style) softmax over the per-frame BiGRU outputs. A linear classifier projects the attention-weighted context vector to the gloss vocabulary. This architecture is referred to throughout the document as the GRU+Attention model and totals approximately 2.26 million parameters at the configurations reported in Section 6.1. Two alternative architectures appear in the comparison tables of Section 6.3: a plain bidirectional GRU baseline and a Transformer encoder with four layers and eight heads.

They share the input pipeline and the loss function with the champion model and differ only in the sequence-modelling block.

Optimisation is performed with AdamW at learning rate 1×10^{-3}, weight decay 5×10^{-4}, and label smoothing 0.1 on the cross-entropy loss. The learning-rate schedule is CosineAnnealingLR. Early stopping monitors the validation Top-1 accuracy with patience 30 epochs and a maximum run length of 150 epochs. Skeletal augmentation during training combines rotation, scaling, translation, additive Gaussian noise, and horizontal flip, applied per clip on the landmark coordinates rather than on pixels.

Evaluation uses two complementary protocols. Stratified 5-Fold cross-validation runs on the WLASL30 vocabulary, with the dataset rebuilt by filtering WLASL100 down to the same 30 classes (332 samples) and the folds drawn so that the per-class distribution is preserved. The multi-seed protocol runs the same configuration with three random seeds (42, 123, 2026) and reports the Top-1, Top-5, Macro-F1, and Balanced Accuracy metrics as mean $\pm$ standard deviation. Test-time augmentation was deliberately excluded from the protocol after supervisory review identified it as a source of biased inflation; the rationale is recorded in Section 6.1.

Reproducibility is a method as much as a property. Every training and evaluation run produces an `experiment_manifest.json` that captures the git commit, the resolved configuration, the hardware identifier, the wall-clock time, and the final metrics. Hyperparameters and dataset paths live in YAML files under `configs/`, never in code, with a base configuration defining the defaults shared across experiments and per-experiment files overriding only the fields they need. The full reproduction procedure is documented in `docs/reproducibility_guide.md`.

The exploratory phase of the project relied on two methods that no longer carry results in the Experiments chapter but are listed here for completeness. The first is YOLOv8n hand detection, used during Phase B (see Section 4.4) to localise hands from raw RGB frames. The second is CNN-based pixel classification of static ASL letters. Both methods reached working accuracy on their respective tasks and both were deprecated when the project pivoted to landmark-based word recognition; Section 6.2 explains why.

4.3 Methodology

The project follows an iterative, experiment-driven methodology rather than a fixed waterfall plan, because the technical direction shifted substantially after the early exploratory phase and any plan that did not allow for that shift would have been discarded in February. The methodology is not borrowed from CRISP-DM, Scrum, or any other process-software framework: those frameworks were considered, but the development log documents a different and simpler pattern, applied repeatedly across the lifetime of the project. That pattern is the basis of the methodology described here.

Each meaningful step of the project is organised as a four-stage loop. The loop starts with a *hypothesis* that is verifiable in principle: for example, "landmark-based representation outperforms pixel-based representation on WLASL30 within the available data budget," or "a sequence length of 64 frames produces lower seed-to-seed variance than 32 or 96." The loop continues with *implementation*: the relevant code changes are made in the `src/slr/` package, augmented with whichever new YAML configuration the experiment requires, and committed to git so that the run is anchored to a specific repository state. The loop then runs the *experiment*: the training script is launched, the manifest is written, and the metrics are persisted alongside the model checkpoints in the experiment

artifact directory. Finally, the loop closes with *reflection*: a devlog entry records what was attempted, what the result was, whether the hypothesis was confirmed, and what the next step should be. If the hypothesis fails, the next iteration starts from a new hypothesis informed by the failure rather than from the original plan.

The pivot recorded in Section 6.2 is the most consequential application of this loop. The project began with a pixel-based pipeline that targeted hand detection with YOLOv8n followed by per-frame letter classification. Hand detection reached approximately 98 % mAP@0.5 on the joint EgoHands+Kaggle test split and was, by any reasonable measure, working. Letter classification was equally tractable on the static-frame benchmarks. The combined pipeline, however, surfaced a structural observation that no incremental tuning would have removed: the bottleneck of the system was not localising the hand but representing the dynamic content of the sign. The four-stage loop responded to that observation by retiring the pixel-based recognition stage and replacing it with the landmark-based word-recognition pipeline that is the subject of every subsequent chapter. The pivot was a methodological consequence, not an arbitrary scope change: the iterative loop is what made the pivot possible, and the development log makes the rationale auditable in retrospect.

Two further properties of the methodology are worth making explicit because they show up repeatedly in the Technical Development and Experiments chapters that follow. The first is the role of tutor meetings as control points. The most impactful technical decisions of the project were made jointly with the supervisor at scheduled meetings rather than unilaterally between iterations. These decisions included removing test-time augmentation from the evaluation protocol after supervisory review, restructuring the State of the Art chapter as a hierarchical CNN→RNN→LSTM→GRU theory progression, adopting Google's AI Principles during the ethics review phase, and splitting the experimental work into quantitative and qualitative tracks during the final experimental review. The meetings act as low-frequency external feedback that the four-stage loop, on its own, would not provide.

The second property is the introduction of the multi-seed protocol described in Section 4.2. The protocol is a direct response to a tutor concern about reproducibility raised in early April 2026: a single-seed Top-1 figure on the 34-sample WLASL30 test split is bounded below in resolution by the granularity of one misclassification (≈2.94 pp), and a single number at that resolution is indistinguishable from initialisation noise. The methodology absorbed the concern by adding a new "aggregate over three seeds" step to every headline experiment going forward and by retroactively applying it to the sequence-length sweep and to MS-ASL50. This is a concrete example of the iterative methodology incorporating supervision: the protocol that distinguishes a defensible thesis number from a noisy point was added in response to a single conversation, propagated through the codebase by adding a small number of YAML files, and reflected in Section 6.1 as the statistical backbone of the experimental claims.

Documentation is treated as a continuous side effect of the loop rather than as a separate phase. The development log contains one entry per meaningful session, dated, scoped to a specific session name, and indexed centrally. The log is the primary source from which the work plan in Section 4.4 is reconstructed; without it, the chronology of the project would be unrecoverable from the code alone.

4.4 Work Plan

The executed work plan was reconstructed from the original proposal, recorded development notes, experiment artifacts, and supervision notes. The plan is organised into seven phases (A–G). The first two phases correspond closely to the proposal-era plan, while the remaining phases reflect the methodological pivot from static, pixel-based recognition to landmark-based word-level ASL recognition.

Phase A: Project framing and proposal. The project formally started in the first week of January 2026 with problem definition, preliminary literature review, scope selection, and proposal preparation. This phase defined the accessibility motivation, the initial system architecture, the first work schedule, and the research direction. No model training was carried out at this stage; the main output was the approved project proposal and its initial planning material.

Phase B: Hand detection and static letter recognition. The initial technical work explored the proposal's first recognition route: localising hands in images and classifying static ASL letters. A YOLOv8n detector was trained on a combined EgoHands and Kaggle Hand Dataset corpus, and CNN-based classifiers were explored on static ASL alphabet datasets. These components reached working performance on their own tasks, but they also revealed a limitation in the original direction: the central research challenge was not hand localisation, but representing the temporal structure of complete signs.

Phase C: Landmark pivot and WLASL30 baseline. In late January and February 2026, the project pivoted from pixel-based letter recognition to word-level recognition using MediaPipe hand landmarks. WLASL30 was built as the first controlled benchmark, and several sequence models were compared under the same landmark input representation. The GRU+Attention architecture emerged as the strongest baseline because it combined a recurrent temporal model with an attention mechanism able to focus on the most informative frames. After supervisory review, test-time augmentation was excluded from the final evaluation protocol, and the WLASL30 result was strengthened through stratified K-fold validation.

Phase D: Vocabulary scaling and ethical framework. Once the GRU+Attention model had been selected, the next phase tested how the approach behaved as the WLASL vocabulary increased beyond the initial 30-class setting. The model was retrained on progressively larger WLASL subsets to quantify the effect of vocabulary size and per-class data scarcity. In parallel, the ethical and social-impact chapter was developed, including the accessibility motivation, privacy implications of landmark-based representations, bias considerations, reproducibility, and the responsible-deployment limits of the prototype.

Phase E: Runtime prototype. In April 2026, the technical work expanded from offline training to a runtime proof of concept. A mobile client and FastAPI server were integrated so that short clips could be captured, sent for inference, and returned as Top-K word predictions. Additional prototype features included text-to-speech, model selection from the client, and a heuristic continuous-prediction demo mode based on overlapping windows. This phase demonstrates end-to-end feasibility, but it is not treated as the main research result; the quantitative contribution remains the landmark-based recognition pipeline and its validation.

Phase F: Refactoring and cross-dataset validation. The notebook-based experimental code was consolidated into a modular Python package, with configuration-driven training, structured experiment manifests, reusable evaluation scripts, and preserved checkpoints and plots. This refactor improved reproducibility and made it possible to rerun the same pipeline across datasets. MS-ASL50 was then added as an independent validation benchmark, and the final evaluation protocol incorporated multi-seed runs, a sequence-length sweep, cross-dataset validation, and comparison against the WLASL results.

Phase G: Documentation and qualitative validation design. The final phase focused on thesis writing, result consolidation, figure and table preparation, and the design of a qualitative validation protocol for the runtime prototype. The experimental chapter was organised into quantitative and qualitative tracks: the quantitative track reports completed model evaluations, while the qualitative track defines the participant protocol and analysis plan for future usability evidence. This phase remains open at the time of this intermediate submission and will be completed with the final thesis revision and qualitative data integration.

Figure 4.1 summarises the executed work plan. The chart follows the seven phases described above and highlights the main deviation from the initial proposal: the pivot from static letter recognition to landmark-based word-level recognition.

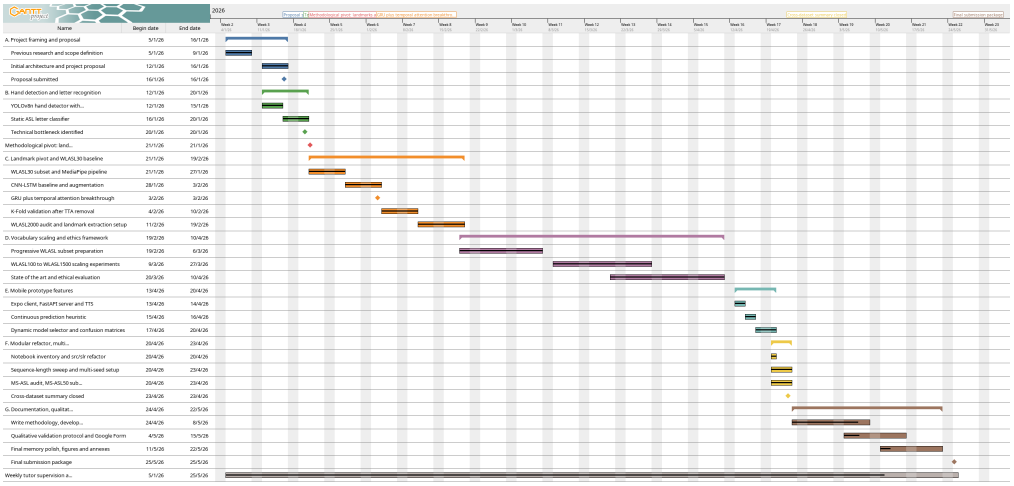


Figure 4.1: Project Gantt chart showing the executed work plan grouped into seven phases, from project framing and initial hand-detection experiments to landmark-based word recognition, cross-dataset validation, prototype integration, and final documentation. Source: self-created with GanttProject.

4.5 Constraints and Limitations

This section catalogues the constraints that shaped the work plan and the methodology rather than the experimental results. Limitations of the experimental conclusions themselves, including standard deviations, test-set granularity, and single-architecture cross-dataset validation, are treated separately in Section 6.8.

The first constraint is time. The project was developed in parallel with a software-engineering internship, which restricted the number of contiguous hours that could be dedicated to it in any given week. The iterative methodology described in Section 4.3 is in part a response to this constraint: a four-stage loop with an explicit devlog entry per session is what makes it possible to resume the project after a multi-day pause without losing the local context. Without the log, the cost of re-entering a problem after an interruption would have dominated the cost of solving it.

The second constraint is hardware. Every training and evaluation run executed on a single NVIDIA RTX 3050 Ti Laptop with 4 GB of VRAM. This ceiling propagated into multiple design choices: `max_frames` = 64 was selected partly because longer sequences exceeded the memory budget at batch size 32, the GRU+Attention model was sized at approximately 2.26 million parameters rather than the parameter counts typical of large Transformers, and the Transformer baseline that does appear in Section 6.3 uses a relatively narrow $d_{\text{model}} = 256$ for the same reason. A cloud GPU was considered and rejected: per-run cost predictability and bit-for-bit reproducibility between consecutive runs were judged more valuable than the additional VRAM.

The third constraint is data availability. Both word-recognition datasets used in the project are reconstructed from external sources rather than distributed directly, and both have lost a measurable fraction of their original clips. WLASL relies on URLs that have decayed since the dataset was published, and the MediaPipe detection rate on the surviving WLASL clips is approximately 69 %, which limits the effective per-clip information content. MS-ASL is reconstructed from per-instance YouTube URLs, and the reconstruction audit records a download success rate of 65.9 % (16,816 of 25,513 clips). The MS-ASL50 subset, rather than the more conventional MS-ASL30, is a direct response to this constraint: only six of the top-30 MS-ASL classes meet the strict 30/5/5 train/val/test threshold from current YouTube state, so an MS-ASL30 reproduction would have been statistically indefensible (cf. Section 6.6).

The fourth constraint is evaluator availability. Until the qualitative validation protocol described in Section 6.9 executes, the author is the only evaluator and the only experimenter on the project. This is a standard configuration for an undergraduate final-degree project and is acceptable for the quantitative work, where the evaluation is fully automated and reproducible from a manifest. It does, however, mean that the cognitive bias of the experimenter is uncontrolled at the design level. The qualitative protocol is structured so that the analysis plan is fixed in advance of data collection, in the spirit of pre-registration, precisely to mitigate this bias once the protocol runs.

The fifth constraint is linguistic scope. The project validates a single sign language, American Sign Language, and uses ASL exclusively for both the WLASL and MS-ASL benchmarks. The pipeline contains no ASL-specific assumptions, and every component that consumes landmarks could in principle be retrained on a different sign language given a comparable benchmark. The validation has not been done, however, and any claim about transfer to Spanish Sign Language, British Sign Language, or any other variety is therefore future work.

The sixth and most general constraint is that the iterative, experiment-driven methodology described in Section 4.3 is, by construction, incompatible with the literal execution of the Gantt that the project proposal recorded in January 2026. The Gantt scheduled hand localisation, static handshape recognition, dynamic sign recognition, temporal fusion and decoding, language generation, mobile prototype, component integration, and validation tests as a sequence of rigid intervals; the methodology actually applied retired the Static Handshape Recognition track in favour of word-level recognition, replaced the proposed PaliGemma language-generation step with a direct gloss-to-text mapping, and folded the Temporal Fusion stage into the GRU+Attention model. This divergence is not a planning

failure but a deliberate property of the methodology. The tutor meeting notes document the moments at which each scope change was approved, and the devlog and meeting notes together act as the audit trail that links the proposal-era plan to the executed plan.

Chapter 5

Technical Development

This chapter describes the engineering side of the project: how the recognition system is structured into independent stages, which functional capabilities each stage provides, in which environment the system runs, and which requirements drove the design. The datasets that feed the pipeline are described in their own section at the end of the chapter.

5.1 System design

The system is structured as a modular three-tier architecture covering data preparation, model training and evaluation, and runtime inference. The three tiers are physically separated (different processes, different machines in the production scenario) but share a single source of truth for data formats and trained model artifacts, so a checkpoint produced in tier two can be served unchanged in tier three. Figure 5.1 shows the overall data flow.

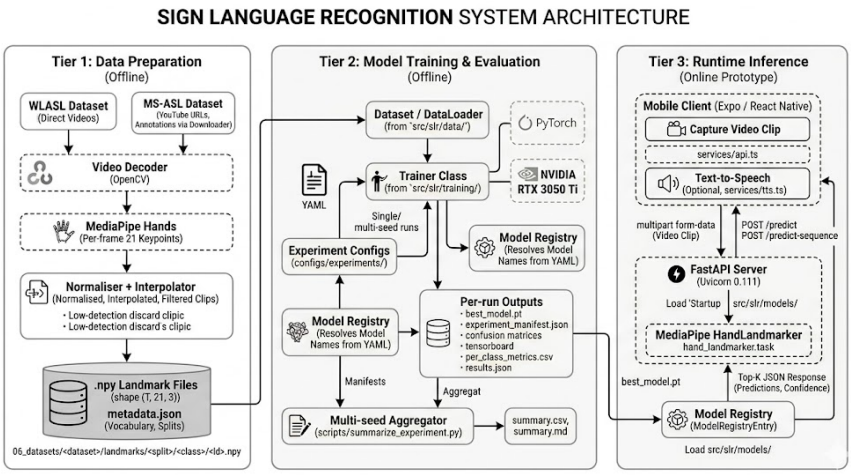


Figure 5.1: System architecture: data preparation, training and evaluation, and runtime inference tiers.

Tier 1: Data preparation pipeline. Raw video material is acquired from the two ASL benchmarks used in the project: WLASL is downloaded directly from the dataset distribution, while MS-ASL is reconstructed by retrieving the original YouTube clips referenced by the dataset annotations and trimming each one with the per-instance `(start_time, end_time)` pair. OpenCV decodes each clip frame by frame, and MediaPipe Hands extracts 21 three-dimensional keypoints per frame. The keypoints are then wrist-centred (the wrist is subtracted from every joint) and length-normalised by the wrist-to-middle-finger distance, so that absolute hand size and frame position no longer carry signal. Frames where MediaPipe fails to detect a hand are linearly interpolated from neighbouring valid frames; clips whose detection rate falls below 0.3 are discarded. Each surviving clip is serialised as a per-clip `.npy` array of shape $(T, 21, 3)$ alongside a `metadata.json` recording the dataset name, gloss vocabulary, split assignments, and the source-video identifiers. This stage is implemented in `src/slr/data/` and exposed through the `scripts/extract_landmarks_*.py` entry points.

Tier 2: Training and evaluation. The Python package `src/slr/` owns the model definitions (`models/` contains `gru_baseline`, `gru_attention`, and `transformer`; a registry resolves the name from a configuration field), the training loop (`training/trainer.py`), the evaluator (`evaluation/`), and the experiment-tracking machinery (`tracking/manifest.py`, `tracking/git_utils.py`). All hyperparameters and dataset paths live in YAML files under `configs/`, never in code; a base configuration (`configs/base/gru_attention.yaml`) declares the defaults shared across experiments, and each experiment YAML overrides only the fields it needs. Every run produces a self-contained directory containing the best checkpoint (by validation Top-1), the training history, confusion matrices (raw and row-normalised), per-class metrics, and an `experiment_manifest.json` that captures git commit, hardware, runtime, hyperparameters, and final metrics. The same trainer supports the multi-seed protocol used to report mean and standard deviation in the experiments chapter.

Tier 3: Runtime inference (prototype). The mobile client lives in `03_app/signtotext-expo/` and is built with React Native and Expo; it captures a short video clip with the device camera and posts it to the inference server through a thin REST client (`services/api.ts`). The server lives in `04_server/main.py`, is implemented with FastAPI, and exposes two endpoints: `POST /predict` for isolated-word inference and `POST /predict-sequence` for a heuristic continuous-sequence variant. On reception, the server runs the same MediaPipe extraction used in tier one, loads the requested model from a curated registry, returns the Top-K predictions with confidence scores, and optionally hands the Top-1 label to a text-to-speech service on the client (`services/tts.ts`). The server reuses the `src/slr/` package directly, so the inference code path is identical to the evaluation code path and no model surgery is needed at deployment time.

5.1.1 Description of functional features

The system exposes six high-level capabilities. Each one is justified by the technical decision it embodies, so this list doubles as a record of why the architecture took its current shape.

- **F1: Landmark-based input pipeline.** Each frame is reduced from a 224×224 RGB tensor to 63 floating-point landmark coordinates, a roughly 2,389$\times$ dimensionality reduction. The choice removes background and skin-tone signal at source, which both shrinks the model and disconnects recognition accuracy from appearance bias.
- **F2: Multi-seed training protocol.** The trainer supports running the same configuration across multiple random seeds and aggregating the metrics with mean and standard deviation. Without this protocol, a single-seed Top-1 figure on a small test set is indistinguishable from noise; with it, a 2–3 percentage-point standard deviation can be reported alongside every headline number.
- **F3: Architecture-agnostic training loop.** The same `Trainer` accepts `gru_baseline`, `gru_attention`, and `transformer` via a registry lookup. This makes the architecture comparison reproducible: only the model field of the YAML changes between runs, every other variable stays fixed.
- **F4: Cross-dataset validation.** The same pipeline runs unmodified on WLASL and on MS-ASL. Only the dataset path and the gloss vocabulary differ between the two configurations; nothing in the model, optimiser, or loss is dataset-specific, which is what makes the cross-dataset comparison fair.
- **F5: Reproducible experiment tracking.** Every run writes an `experiment_manifest.json` that captures git commit, dataset metadata, hyperparameters, hardware, runtime, and final metrics in one place. A future user can reconstruct the exact configuration of any historical run from this single file.
- **F6: Top-K predictions in inference.** The runtime API returns Top-1 and Top-5 candidates with confidence scores instead of forcing a single answer. This matches the way the model itself is evaluated and lets the client surface a shortlist when the Top-1 confidence is low.

5.1.2 Operational Environment

Training and landmark extraction were performed on the local development workstation; the runtime prototype was tested with a paired mobile client and an HTTP server. Table 5.1 lists the components

of the operational environment and the version constraints under which the results in this thesis were produced.

Table 5.1: Operational environment for training, landmark extraction, and runtime inference.

Component	**Specification**
Operating system	Ubuntu 24.04 LTS (training and server); cross-platform (mobile client).
CPU	Multi-core x86_64 (local laptop).
GPU	NVIDIA RTX 3050 Ti Laptop, 4 GB VRAM, CUDA 12.x.
Python	3.10 (project); 3.11 supported via `requires-python = ">=3.10"`.
Deep learning framework	PyTorch 2.x with CUDA support.
Pose estimation	MediaPipe Hands 0.10+ (`HandLandmarker`).
Video processing	OpenCV 4.8+.
Mobile client framework	React Native + Expo.
Backend	FastAPI 0.111 with Uvicorn 0.29.
Persistence	JSON manifests, `.npy` landmark files, `.pt` PyTorch checkpoints.
Version control	Git.

The development environment is reproducible through the package metadata declared in `pyproject.toml` (editable install with `pip install -e .[landmarks]`) and the dedicated server requirements declared in `04_server/requirements.txt`. The end-to-end reproduction procedure for the headline experiments, including environment setup, dataset preparation, training commands, and evaluation, is documented in `docs/reproducibility_guide.md`.

5.2 Requirements

5.2.1 Description of requirement attributes

For the textual description of the different identified requirements, a set of attributes has been selected to describe each requirement. Below is a description of the meaning of each attribute used:

- **Code**: A unique abbreviated identifier for the requirement. It is constructed using the requirement type code followed by a hyphen and three digits. Requirements are divided into functional and non-functional requirements. The codes used are **FR** for Functional Requirements and **NFR** for Non-Functional Requirements. For example: `FR-001`.
- **Name**: Extended identification of the requirement.
- **Description**: A basic description of the identified requirement.
- **Source**: Indicates the source through which the requirement was identified. This value usually corresponds to one or more use case codes.
- **Necessity**: Determines the degree of implementation required for the requirement. The possible values are:

  - **Essential**: The requirement must be implemented.
  - **Desirable**: It is preferable to implement the requirement, but it is not mandatory.
  - **Optional**: The requirement may be implemented, but it is neither important nor mandatory.
- **Priority**: Defines the importance of the requirement, allowing the order in which requirements are included in the design and implementation process to be established. The possible values are:
  - **High**: The requirement must be implemented in the early stages of development.
  - **Medium**: The requirement should be implemented once high-priority requirements have been completed.
  - **Low**: The requirement should be implemented in the final stages of development. These requirements do not affect the correct functioning of the system.
- **Stability**: Defines the stability of the requirement throughout the software life cycle. This indicates whether the requirement may be modified during the life cycle. The possible values are:
  - **Stable**: The requirement will not change during the system life cycle.
  - **Unstable**: The requirement may change during the system life cycle.
- **Verifiability**: Defines the degree to which a requirement can be verified, that is, the extent to which it is possible to confirm that the requirement has been implemented in the developed system. The possible values are:
  - **High**: It is possible to verify that the requirement has been implemented. These requirements usually correspond to core system functionalities.
  - **Medium**: It is possible to verify that the requirement has been implemented, but it requires complex testing or inspection of the source code.
  - **Low**: It is difficult, or in some cases impossible, to verify whether the requirement has been implemented.

5.2.2 Functional Requirements

The system specifies four functional requirements covering the core recognition pipeline (FR-001, FR-002), the experiment-tracking infrastructure (FR-003), and statistical validation through multi-seed training (FR-004).

System requirement			
ID	FR-001	**Source**	Pipeline design
Name	Landmark extraction from video clips		
Description	The system shall extract 21 three-dimensional hand landmarks per frame from each input video clip using MediaPipe Hands, producing a per-clip array of shape $(T, 21, 3)$ stored as a `.npy` file alongside dataset metadata.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.2: FR-001: Landmark extraction from video clips

System requirement			
ID	FR-002	**Source**	Project objective
Name	Word-level sign classification		
Description	The system shall classify input video clips into one of the N predefined ASL word classes for the active dataset, returning Top-1 and Top-5 predictions accompanied by confidence scores.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.3: FR-002: Word-level sign classification

System requirement			
ID	FR-003	**Source**	Reproducibility requirements
Name	Reproducible experiment tracking		
Description	Every training run shall generate a structured manifest capturing dataset version, hyperparameters, git commit, hardware, runtime, and final metrics in a single JSON file, sufficient to reconstruct the run configuration without consulting external sources.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.4: FR-003: Reproducible experiment tracking

System requirement			
ID	FR-004	**Source**	Statistical validation
Name	Multi-seed training support		
Description	The training subsystem shall support running the same configuration across multiple random seeds and aggregating the resulting metrics with mean and standard deviation, so that every reported result can be accompanied by an uncertainty estimate.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	High

Table 5.5: FR-004: Multi-seed training support

5.2.3 Non-functional Requirements

Three non-functional requirements characterise quality attributes of the system: runtime performance (NFR-001), code quality and modularity (NFR-002), and documentation completeness (NFR-003).

System requirement			
ID	NFR-001	**Source**	User experience
Name	Inference latency		
Description	End-to-end inference for a single video clip shall complete in under two seconds on the target deployment hardware (mobile client recording the clip, server running landmark extraction and model prediction).		
Necessity	Desirable	**Priority**	Medium
Stability	Unstable	**Verifiability**	Medium

Table 5.6: NFR-001: Inference latency

System requirement			
ID	NFR-002	**Source**	Maintainability
Name	Modular code architecture		
Description	The codebase shall be organised as a Python package with separate submodules for data, models, training, evaluation, tracking, and utilities, supporting independent unit testing of each submodule.		
Necessity	Essential	**Priority**	High
Stability	Stable	**Verifiability**	High

Table 5.7: NFR-002: Modular code architecture

System requirement			
ID	NFR-003	**Source**	Academic rigor
Name	Documentation completeness		
Description	The repository shall include a reproducibility guide that enables a third party to replicate the headline experimental results with documented commands and dependency declarations.		
Necessity	Essential	**Priority**	Medium
Stability	Stable	**Verifiability**	Medium

Table 5.8: NFR-003: Documentation completeness

5.3 Datasets

This project spans two recognition tasks with fundamentally different data requirements: hand localization from still images, and word-level sign language recognition from video. As a consequence, six distinct datasets are employed across the full experimental pipeline. This section formalizes the properties of each dataset by first defining a common set of descriptive attributes, then presenting a unified comparative table, and finally providing individual descriptions of each dataset in context.

5.3.1 Dataset Attribute Definitions

To ensure a consistent and unambiguous characterization of every dataset, the following attributes are used in Table 5.9. Each attribute captures a distinct dimension relevant to dataset selection, reproducibility, and the interpretation of experimental results:

- **ID**: A unique alphanumeric identifier assigned to each dataset within this thesis. The prefix `DS-` followed by a two-digit index allows unambiguous cross-referencing in subsequent chapters when reporting experimental configurations and results.
- **Name**: The official or widely-used name of the dataset as it appears in the literature or on its hosting platform.
- **Year**: The year of original publication or first public release. This contextualizes the dataset within the chronological development of the field and helps evaluate whether it reflects the current state of the art in data collection methodology.
- **Task**: The specific sub-task for which the dataset was designed. Three task categories appear in this project:
  - **Hand Detection**: the dataset provides annotated bounding boxes or segmentation masks marking the location of hands in images.
  - **Letter Recognition**: the dataset provides labeled static images of individual American Sign Language (ASL) letter signs.
  - **Word Recognition**: the dataset provides labeled video clips of complete ASL word signs, requiring temporal modeling.
- **Samples**: The total number of annotated data instances. For image datasets, this equals the number of images; for video datasets, it equals the number of individual video clips. Where a subset of the full dataset was used in this project, this is noted in the individual descriptions.
- **Classes**: The number of distinct output categories. For hand detection datasets this is always 1 (the single *hand* class); for sign language datasets it corresponds to vocabulary size, meaning the number of distinct letters or word glosses.
- **Format**: The storage format and annotation type of the raw data. This attribute specifies whether instances are static images or video clips, the image resolution where relevant, and the form of ground-truth annotation (e.g., bounding box coordinates in XML, pixel-level segmentation masks, image class labels, or video-level class labels).
- **Input Representation**: Whether the models trained on this dataset consume *pixel-based* data (raw RGB or grayscale image/video frames) or *landmark-based* data (structured 3D keypoint coordinates extracted by a pose estimator such as MediaPipe). Pixel-based inputs preserve full visual detail but are high-dimensional and sensitive to background, lighting, and signer appearance. Landmark-based inputs are compact, viewpoint-invariant, and more robust to visual variability, at the cost of depending on a reliable keypoint extractor.

5.3.2 Comparative Overview

Table 5.9 provides a high-level comparison of the six datasets used in this project. Datasets DS-01 and DS-02 support the hand localization pipeline described in later experimental sections. Datasets DS-03 and DS-04 were used in the initial letter-level classification phase of the project, which was

subsequently superseded by the word recognition approach. Datasets DS-05 (WLASL) and DS-06 (MS-ASL) are the primary benchmarks for word-level recognition. WLASL is the dataset on which the architecture was tuned; MS-ASL was added in a later phase to validate that the same pipeline transfers to an independent ASL corpus with a different signer distribution.

Table 5.9: Comparative overview of all datasets used in this project.

ID	Name	Year	Task	Samples	Classes	Format
DS-01	EgoHands	2015	Hand Detection	4,800	1	Images + Seg. Masks
DS-02	Kaggle Hand Dataset	2023	Hand Detection	∼4,200	1	Images + BBox (XML)
DS-03	Sign Language MNIST	2017	Letter Recog.	34,627	24	28×28 Grayscale CSV
DS-04	ASL Alphabet	2018	Letter Recog.	87,000	29	200×200 RGB JPG
DS-05	WLASL	2020	Word Recog.	21,083	2,000	Video Clips (RGB)
DS-06	MS-ASL	2018	Word Recog.	25,513	1,000	Video Clips (RGB, YouTube)

5.3.3 Hand Detection Datasets

Datasets DS-01 and DS-02 are used jointly to train the hand localization module based on YOLOv8. A central design motivation for combining two datasets is improved generalization: a detector trained exclusively on egocentric footage tends to fail when the camera is positioned at eye level or beyond arm's reach, while a detector trained only on third-person images may struggle with first-person perspectives. By merging the two sources, the training corpus covers a significantly wider range of viewpoints, backgrounds, and hand appearances. After merging, duplicate removal, and a stratified train/validation/test split, the combined training set comprises approximately 5,223 annotated images.

DS-01: EgoHands [28]

EgoHands is an egocentric hand detection dataset published by Indiana University in 2015 as part of research on activity recognition in close-proximity social interactions. It consists of approximately 4,800 labeled frames extracted from 48 video recordings captured from a GoPro camera mounted on the chest of an observer watching another person. Ground-truth annotations are stored as pixel-level segmentation polygons in MATLAB `.mat` files; during preprocessing, these polygons were converted to axis-aligned bounding boxes in YOLO format. The dataset covers diverse interaction scenarios, including card games, chess, and jigsaw puzzles, with two hands simultaneously visible in most frames, providing challenging overlap and occlusion cases. Because all images are taken from a first-person perspective, the visual distribution of EgoHands is inherently biased toward the observer's own hand. This limitation motivates the inclusion of DS-02 to extend viewpoint diversity. The input representation is pixel-based: models consume raw RGB crops.

DS-02: Kaggle Hand Dataset [29]

The Kaggle Hand Dataset is a collection of annotated hand images published on the Kaggle platform, providing multi-viewpoint coverage that is complementary to the egocentric nature of DS-01. Images depict hands in third-person perspectives with diverse backgrounds, lighting conditions, skin tones, and partial occlusions. Ground-truth bounding boxes are provided in Pascal VOC XML format; during preprocessing, coordinates were normalized to the YOLO format used by the detection pipeline. The dataset contains approximately 4,200 labeled images. The joint training corpus formed by DS-01 and DS-02 yielded a YOLOv8n model achieving approximately 98% mAP50 and 87% Mean IoU on the held-out test split, demonstrating the effectiveness of the multi-source approach. The input representation is pixel-based.

5.3.4 Letter Recognition Datasets

Datasets DS-03 and DS-04 were used during the initial phase of the project, when the recognition target was the static letter-level ASL alphabet. Both datasets cover 24 of the 26 ASL letters; the letters *J* and *Z* are excluded in both cases because they require hand motion and cannot be represented by a single static image. The two sources were merged and unified into a single training corpus for letter classification experiments. Although this phase was later superseded by the more ambitious word-level recognition approach, these datasets remain relevant because they established the viability of data-driven letter classification and informed the design of the preprocessing pipeline. The input representation for both datasets is pixel-based.

DS-03: Sign Language MNIST [30]

Sign Language MNIST mirrors the format of the canonical MNIST handwritten digit benchmark. It contains 27,455 training images and 7,172 test images, for a total of 34,627 samples, each representing one of 24 static ASL letters as a 28×28 grayscale image. Data is distributed in CSV format, with one row per image where each column corresponds to a pixel intensity value in the range $[0, 255]$, plus a label column. The low resolution and grayscale format make this dataset extremely lightweight and straightforward to preprocess, but inherently limit the discriminative information available to the model, as fine-grained finger articulation details are largely lost. The dataset was originally constructed from ASL alphabet photographs rescaled and converted to the MNIST-style format, and is hosted on Kaggle.

DS-04: ASL Alphabet [31]

The ASL Alphabet dataset provides a substantially richer visual input than DS-03. It contains approximately 87,000 images organized into 29 classes: the 24 static ASL letters, the letter *space*, and two additional non-letter classes (*del*, *nothing*). Each class is represented by exactly 3,000 training images captured in a controlled environment at a resolution of 200×200 pixels in full RGB color. The higher resolution and color information enable more discriminative feature learning, particularly for hand texture and illumination cues. For integration with DS-03 into a unified corpus, images were converted to grayscale and resized to a common format, yielding a combined training set of approximately 79,000 images and 7,000 test images covering the 24 shared letter classes.

5.3.5 Word Recognition Dataset

DS-05: WLASL [1]

The Word-Level American Sign Language (WLASL) dataset, published by Li et al. in 2020, is the primary data resource for all word-level recognition experiments in this thesis and one of the largest publicly available ASL word recognition benchmarks to date. WLASL contains 21,083 video instances covering 2,000 ASL glosses (words). Videos were collected from multiple online platforms, including YouTube channels maintained by ASL educators, online courses, and signing dictionaries, and manually verified by qualified ASL experts to ensure correctness of sign production. Each clip shows a single signer performing one complete sign, typically against a plain or low-clutter background, with durations ranging from approximately one to four seconds.

The dataset is partitioned into four progressively larger evaluation subsets: WLASL100 (100 glosses), WLASL300 (300 glosses), WLASL1000 (1,000 glosses), and WLASL2000 (the full 2,000 glosses). This structure enables systematic benchmarking of how model performance scales with vocabulary size. A practical challenge is that WLASL relies on external video URLs; a fraction of clips have become unavailable over time due to deletion or access restriction, so only successfully downloaded videos can be used.

In this project, raw RGB frames are not used as model input. Instead, MediaPipe Hands [8] is applied to each frame to extract 21 three-dimensional hand keypoints, yielding a compact landmark vector of 63 scalar features per frame (21 joints $\times$ 3 coordinates). This representation reduces input dimensionality by a factor of approximately 2,389 compared to raw 224×224 RGB frames, removes background and appearance noise, and provides geometric invariance to signer appearance. The effective detection rate of MediaPipe on WLASL clips is approximately 69%; frames where no hand is detected are linearly interpolated from adjacent detected frames to maintain sequence continuity. Each video is resampled to a fixed sequence length of 64 frames before being fed to the model.

For the experiments described in this thesis, the following WLASL subsets were extracted and used:

- **WLASL30**: The 30 most frequent glosses, used for initial architecture exploration and model selection. Split: 242 training / 56 validation / 34 test videos.
- **WLASL100**: The 100 most frequent glosses, used for K-Fold cross-validation and robust evaluation. Split: 741 training / 162 validation / 99 test videos.
- **WLASL300, WLASL1000, WLASL2000**: Larger subsets targeted for progressive scaling experiments in later stages of the project, where the goal is to assess model accuracy and generalization at increasing vocabulary sizes.

DS-06: MS-ASL [2]

The Microsoft American Sign Language (MS-ASL) dataset, published by Vaezi Joze and Koller in 2018, is a second large-scale ASL benchmark used in this project to validate that the recognition pipeline tuned on WLASL transfers to an independent corpus with a different signer distribution. The original release defines 25,513 video instances over 1,000 glosses, split into 16,054 train / 5,287 validation / 4,172 test samples. Clips are not distributed directly: the dataset ships only annotation JSONs and relies on per-instance YouTube URLs plus `(start_time, end_time)` timestamps and a normalized signer bounding box. The vocabulary is organized into four cumulative subsets (MS-ASL100/200/500/1000) obtained by filtering on the integer label.

Acquiring the raw clips is therefore a non-trivial preprocessing step. Of the 25,513 declared instances, **16,816 were successfully downloaded (65.9% coverage)**, with split-level coverage of 74.0% / 47.5% / 58.0% (train / val / test). The shortfall is caused by YouTube videos being removed or made private over time and is the dominant constraint on which subsets can be reproduced today. A disk-level audit then graded every downloaded clip by integrity and by training viability. Only **6** of the top-30 classes met the strict MS-ASL30 threshold of 30 train / 5 val / 5 test surviving samples; **17** met the MS-ASL50 threshold of 20 / 5 / 5; and **57** met the MS-ASL100 threshold of 20 / 3 / 3. The strict MS-ASL30 cut would discard 24 of the 30 nominal classes, so an MS-ASL30 reproduction was not viable. The chosen alternative is **MS-ASL50**, defined as the 50 classes with the largest train counts that satisfy the 20 / 3 / 3 threshold; this preserves the size-of-vocabulary objective while keeping every class statistically defensible.

The MS-ASL50 subset was processed with the same MediaPipe pipeline used for WLASL: each frame is reduced to 21 three-dimensional hand keypoints (63 features) and stored as a per-clip `.npy` array. After extraction, the final per-split counts are **1,410 train / 306 val / 236 test (1,952 total)**, with 31 source clips dropped because their mean detection rate fell below the 0.3 floor. The mean detection rate across the surviving clips is **83.9%**, substantially above the WLASL figure, consistent with MS-ASL clips being longer and shot under more controlled conditions. Apart from the change of dataset, the GRU + Temporal Attention model consumes the splits without any architectural modification.

Chapter 6

Experiments

This chapter presents the experimental work in the order in which it actually unfolded during the project, from the early hand-detection and letter-classification probes that motivated the methodological pivot, through the architecture comparison and hyperparameter sweep that produced the champion configuration, to the multi-seed and cross-dataset validation that closed the loop. Every result reported below is backed by stored experiment artifacts, including configurations, metrics, plots, trained checkpoints, and an experiment manifest for each run. The detailed repository paths needed to reproduce or audit the results are centralised in the reproducibility guide and internal audit material, so the chapter cites experiment outputs conceptually rather than repeating long file paths in every caption.

6.1 Experimental Methodology

The experimental design rests on two complementary protocols. The first is a **multi-seed protocol** that runs every headline configuration with three random seeds (42, 123, 2026) and reports metrics as mean $\pm$ standard deviation. The motivation is statistical: the WLASL30 test split contains only 34 samples, so a single misclassification moves Top-1 by approximately 2.94 percentage points. A single-seed Top-1 figure on this test set is therefore indistinguishable from initialisation noise, and any claim about the system's behaviour requires aggregating across seeds. The second protocol is **Stratified 5-Fold Cross-Validation** on WLASL100 filtered to the same 30 classes (332 total samples, 5 non-overlapping folds with class distribution preserved); it complements the multi-seed protocol by varying the test split itself rather than only the model initialisation.

Four metrics are reported throughout the chapter. **Top-1 accuracy** is the proportion of test samples for which the highest-scoring prediction matches the ground truth and is the headline figure most often quoted. **Top-5 accuracy** is the proportion of test samples for which the ground truth appears anywhere in the five highest-scoring predictions; it matters in this domain because a real-time captioning system can present a shortlist when the Top-1 confidence is low. **Macro-F1** is the unweighted mean of per-class F1, which exposes degradation on rare classes that Top-1 averages would hide. **Balanced accuracy** averages per-class recall, providing a complementary view of class-imbalance effects.

All training runs share the same hardware and optimisation schedule. Training and evaluation ex-

ecute on a local workstation with an NVIDIA RTX 3050 Ti Laptop GPU (4 GB VRAM, CUDA 12.x). The optimiser is AdamW with learning rate 1×10^{-3}, weight decay 5×10^{-4}, and label smoothing of 0.1 on the cross-entropy loss. The learning rate follows a CosineAnnealingLR schedule. Early stopping monitors validation Top-1 with patience 30 epochs, and the maximum run length is 150 epochs. These values are fixed across every experiment in this chapter unless explicitly noted otherwise.

Test-Time Augmentation (TTA) was deliberately excluded from the evaluation protocol. Earlier exploratory work compared TTA-enabled and TTA-disabled runs on WLASL30 and found that TTA inflates the reported Top-1 without producing a corresponding gain in true generalisation. The decision is methodological rather than incidental: every number in this chapter reflects a single forward pass per test sample.

Part I: Quantitative Evaluation

6.2 Phase 1: Exploratory Work and Methodological Pivot

The first months of the project explored two avenues that were eventually superseded by the landmark + word approach. They are reported here because the lessons drawn from them are the technical justification of the pivot, not because they appear as headline numbers in the final system.

Hand detection with YOLOv8n. An initial sub-system trained YOLOv8n [19] on the joint EgoHands and Kaggle Hand Dataset corpus described in Section 5.3.3 (approximately 5,223 annotated images after stratified train/val/test split and duplicate removal). The model reached approximately **98 % mAP@0.5** and a **Mean IoU of approximately 87 %** on the held-out test split, as verified by the detector evaluation outputs. YOLOv8n was selected after a wider comparison against SSDLite-MobileNetV3 and EfficientDet-D0; the YOLO family won on inference latency at comparable detection accuracy on this dataset.

Letter-level pixel classification. A parallel track trained CNN classifiers on the union of Sign Language MNIST (DS-03) and ASL Alphabet (DS-04). Coverage was limited to 24 of the 26 ASL letters because *J* and *Z* require hand motion and cannot be represented by a single static image. Accuracy on this static-frame classification task was high, but the task was also far simpler than the eventual project goal: predicting one of $\sim$24 hand poses per frame is a different problem from predicting a word from a multi-frame sign.

What the exploration revealed. The bounding-box detector was not the bottleneck, since it worked reliably. The bottleneck was *representation*. Pixel-based recognition has three properties that are problematic at the WLASL scale: (i) it carries appearance signal (skin tone, lighting, background) that is irrelevant to the linguistic content of a sign and that the model has to learn to discount with limited data; (ii) the parameter cost of an image-conditioned sequence model grows quickly, which makes overfitting on a few hundred clips per class very likely; and (iii) the geometry of hand articulation, which is the actual carrier of meaning, is not made explicit to the model.

The pivot to landmark-based word recognition replaced the visual problem with a geometric one: each frame is reduced from a 224×224 RGB tensor to 21 three-dimensional keypoints (63 features), a reduction of approximately $2{,}389\times$ in input dimensionality. The representation discards

background and appearance at the source and exposes the joint geometry directly to a temporal model. The remainder of this chapter studies the resulting pipeline.

6.3 Architecture Selection on WLASL30

WLASL30 (the 30 most frequent WLASL classes; 242 train / 56 validation / 34 test clips) was used as a controlled bench to compare candidate architectures under matched conditions. All five rows in Table 6.1 share the same dataset, the same input pipeline, the same hardware, and the same hyperparameters from Section 6.1, with the exception of the Transformer row whose configuration uses $d_{\text{model}} = 256$, $\text{nhead} = 8$, four transformer layers, $\text{lr} = 5 \times 10^{-4}$, $\text{wd} = 1 \times 10^{-4}$, and dropout 0.3, as recorded in the corresponding experiment manifest.

Table 6.1: Single-split test results of the five architectures evaluated on WLASL30 (34 test samples). Source: self-created from experiment outputs. Standard deviations are not available at this stage; the multi-seed validation of the champion is in Section 6.5.

Architecture	**Top-1 (%)**	**Top-5 (%)**	**Notes**
LSTM (landmarks)	41.18	67.65	Plain LSTM baseline.
GRU (landmarks)	38.24	76.47	Plain bidirectional GRU.
GRU + Enriched features	47.06	79.41	Bone vectors and joint-angle augmentations.
GRU + Attention	**58.82**	**91.18**	Champion; 2.26 M parameters.
Transformer	50.00	73.53	Four layers, 8 heads, $d_{\text{model}} = 256$.

The ordering in Table 6.1 is consistent with the data regime. Plain GRU and plain LSTM are both starved for training data at 242 examples spread over 30 classes; with a comparable parameter budget, the GRU's lower count reduces overfitting at the cost of some ceiling. Adding hand-engineered enriched features (bone vectors, inter-joint angles) gives the model a 9-point absolute Top-1 lift, but the gain disappears once the model is given a Temporal Attention layer that can learn the discriminative time slices on its own. The Attention head dominates because sign language clips contain a small number of *key frames* where the linguistic content is concentrated, surrounded by neutral preparation and recovery frames; weighting the GRU outputs by a learned attention distribution focuses capacity where it matters and tolerates variation in execution speed across signers.

The Transformer row is the most pedagogically interesting. With a parameter budget that is competitive with the GRU+Attention model, the Transformer underperforms on Top-1 (50.00 vs. 58.82) and underperforms more sharply on Top-5 (73.53 vs. 91.18). The result is consistent with the observation in the literature that recurrent inductive biases are an asset in low-data regimes; the Transformer's lack of a built-in sequential prior would plausibly be amortised at WLASL2000 scale, but the experiment in this section does not provide that scale.

The champion configuration, a bidirectional GRU with hidden size 256, two layers, dropout 0.4, followed by a Temporal Attention head, is therefore selected as the default for every downstream experiment in the chapter. Its confusion matrix on the WLASL30 test split is shown in Figure 6.1.

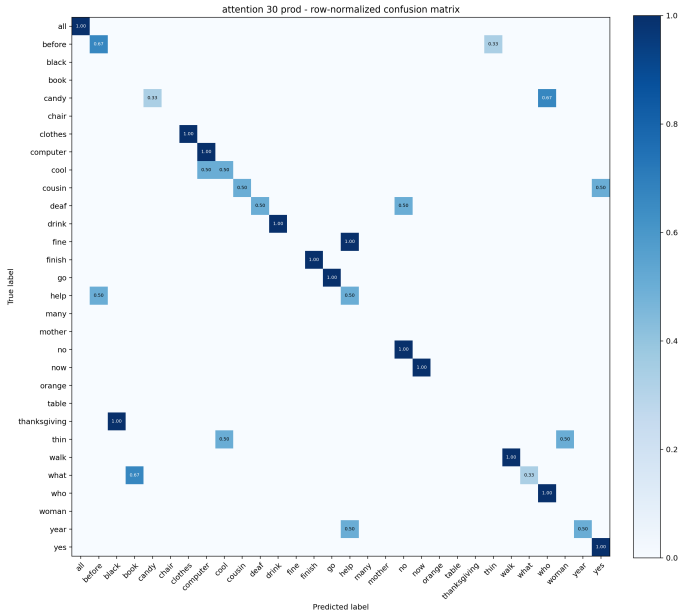


Figure 6.1: Row-normalised confusion matrix of the GRU+Attention champion on the WLASL30 test split (34 samples). Source: self-created from experiment outputs.

The matrix exposes two structural failure modes. The first is class imbalance at evaluation time: several test classes have only one or two samples, so a single error is enough to drop their per-class recall to zero. The second is genuine visual similarity between specific signs: according to the stored experiment outputs, the most common confusions on this split are *candy* $\rightarrow$ *who* and *what* $\rightarrow$ *book*, both at 2 of 3 affected samples, plus single-sample failures on *fine* $\rightarrow$ *help*, *thanksgiving* $\rightarrow$ *black*, and *cool* $\rightarrow$ *computer*.

6.4 Sequence Length Sweep

The maximum sequence length `max_frames` is the only hyperparameter that the model sees as a structural input shape rather than as a regulariser, so its value has to be defended explicitly. Subsampling too aggressively (32 frames) discards temporal dynamics; oversampling (96 frames) introduces padding and noise on shorter clips. A multi-seed sweep distinguishes signal from initialisation variance.

The sweep covers nine runs: 3 seeds (42, 123, 2026) $\times$ 3 sequence lengths (32, 64, 96), with every other variable held fixed at the values in Section 6.1. Aggregated results from the sweep are reported in Table 6.2.

Table 6.2: Sequence-length sweep on WLASL30 with 3 seeds per row. Mean ± standard deviation across seeds. Top-1 standard deviations are bounded below by the test-set granularity (≈2.94 pp per misclassification on 34 samples). Source: self-created from experiment outputs.

Frames	Top-1	Top-5	Macro-F1	Balanced Acc.
32	56.86 ± 6.12	85.29 ± 2.95	50.52 ± 8.27	64.65 ± 7.48
64	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76
96	54.90 ± 4.49	81.37 ± 4.49	49.63 ± 6.33	61.36 ± 6.56

The 64-frame configuration produces the most stable training across the three seeds. Macro-F1 standard deviation drops to 0.93 pp, and Balanced Accuracy standard deviation to 0.76 pp, both an order of magnitude tighter than at 32 or 96 frames. The sweep does not select the configuration that maximises any single mean metric, since 32 frames actually edges out 64 on Top-5 (85.29 vs. 78.43), but it does select the configuration whose mean is least sensitive to the seed. The selection criterion is reproducibility, not peak Top-1, and that is the criterion this chapter applies to every downstream choice.

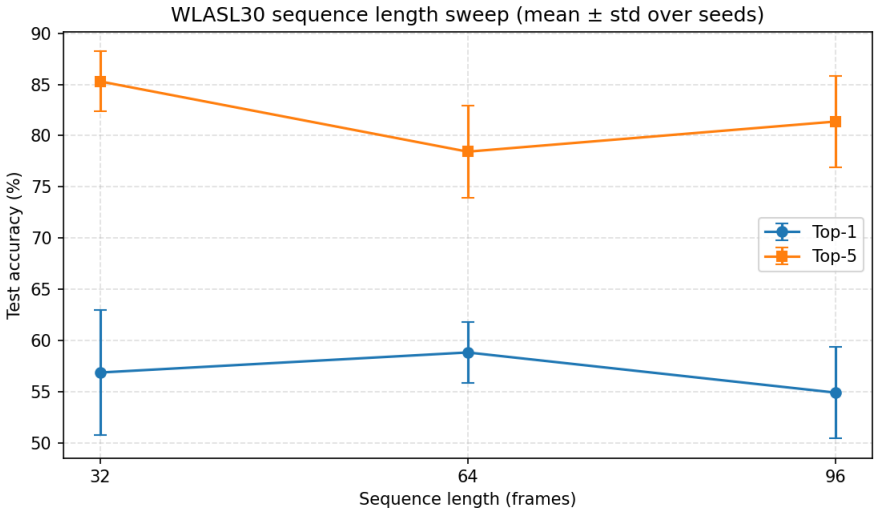


Figure 6.2: Per-metric distribution across the sequence-length sweep (3 seeds at 32, 64, and 96 frames). Source: self-created from experiment outputs.

The 64-frame value is therefore fixed as the default for every downstream experiment in this chapter, including the cross-dataset MS-ASL50 evaluation in Section 6.6. A dedicated sweep on MS-ASL50, where median clip length is approximately 80 frames, would be a natural extension and is acknowledged in Section 6.8.

6.5 WLASL30 Validation

Two independent validation protocols characterise the champion model on WLASL30. The first is the **Stratified 5-Fold Cross-Validation** run on WLASL100 filtered to the same 30 classes (332 total samples), which yields a Top-1 mean of **63.85 % ± 4.50 %** and a Top-5 mean of **86.46 % ± 2.04 %**. This is the headline thesis number because it averages across both initialisation and split, and because the standard deviations are computed across non-overlapping test partitions. The second is the **multi-seed evaluation** on the fixed WLASL30 split at $max_frames = 64$ extracted from the sequence-length sweep (Table 6.3).

Table 6.3: Per-seed test results of the GRU+Attention champion on the fixed WLASL30 split (34 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Acc.	Best Epoch
42	55.88	73.53	55.94	68.18	40
123	61.76	79.41	54.20	66.67	29
2026	58.82	82.35	54.49	67.42	27
Mean ± Std	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93	67.42 ± 0.76	N/A

The two protocols disagree by approximately five percentage points on Top-1 (63.85 vs. 58.82). The disagreement is informative rather than contradictory: 5-Fold CV varies the test partition itself, so the Top-1 average is taken over five non-overlapping test partitions of approximately 66 samples each ($332/5 = 66.4$) and effectively smooths the worst single split; the multi-seed protocol fixes the 34-sample test split and varies only the model initialisation and batch order. Both quantities are real; they answer different questions. The headline number reported elsewhere in the thesis is the 5-Fold mean because it is the more statistically robust of the two and is directly comparable to the K-Fold standard in the SLR literature.

The per-class behaviour of the champion on the fixed split is concentrated on a small number of low-support classes. According to the per-class metric outputs, the worst recall values among supported classes are *fine* (1 sample, recall 0.00), *thanksgiving* (1, 0.00), *thin* (2, 0.00), *candy* (3, 0.33), *what* (3, 0.33), and *cool* (2, 0.50). With one to three test samples per class, even a single misprediction yields a 33–100 pp recall drop. This is a property of the test split, not of the model: the per-class metric can be recovered only by enlarging the evaluation set, which is exactly what the cross-dataset experiment in Section 6.6 achieves.

6.6 Cross-Dataset Validation: MS-ASL50

The pipeline was tuned on WLASL30. To check whether its behaviour is a property of the method or of the benchmark, the same code path was retrained on MS-ASL50 with no architectural changes and no dataset-specific hyperparameter tuning. The only fields that differ between the two configurations are the dataset path and the gloss vocabulary.

6.6.1 Building the MS-ASL50 subset

MS-ASL ships only annotation JSONs and YouTube URLs; the 25,513 declared instances must be retrieved per-clip, and not all of them are still online. The dataset reconstruction audit records that **16,816 of 25,513 clips were successfully downloaded (65.9 %)**, with split-level coverage of 74.0 % / 47.5 % / 58.0 % (train / val / test). The drop-out is dominated by YouTube videos that have been removed or made private since the dataset was published.

The subset selection itself is constrained by data deficiency. The disk-level audit reports that only **6** of the top-30 classes meet the strict 30 train / 5 val / 5 test threshold; **17** meet the 20 / 5 / 5 threshold; and **57** meet the relaxed 20 / 3 / 3 threshold. An MS-ASL30 reproduction is therefore not viable from the locally available data. The chosen alternative is **MS-ASL50**, defined as the 50 classes with the largest train counts among those satisfying the 20 / 3 / 3 threshold. After landmark extraction, 31 source clips are dropped because their mean MediaPipe detection rate falls below the 0.3 floor, leaving **1,410 train / 306 val / 236 test (1,952 total)**. The mean detection rate across surviving clips is **83.9 %**, substantially above the WLASL figure (approximately 69 %), consistent with MS-ASL clips being longer and shot under more controlled conditions.

6.6.2 Cross-dataset results

Both datasets were trained with three seeds (42, 123, 2026). Aggregated metrics are in Table 6.4, with the MS-ASL50 per-seed breakdown in Table 6.5.

Table 6.4: Side-by-side comparison of the GRU+Attention pipeline on WLASL30 and MS-ASL50. Both rows are aggregated over 3 seeds. Source: self-created from experiment outputs.

Dataset	Classes	Train / Val / Test	Top-1	Top-5	Macro-F1
WLASL30	30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	54.88 ± 0.93
MS-ASL50	50	1,410 / 306 / 236	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44

Table 6.5: Per-seed test results on MS-ASL50 (236 test samples). Source: self-created from experiment outputs.

Seed	Top-1	Top-5	Macro-F1	Balanced Acc.	Best Epoch
42	66.95	89.83	64.22	66.34	55
123	72.46	90.25	70.81	72.75	120
2026	71.19	91.53	69.21	71.58	92
Mean $\pm$ Std	70.20 ± 2.89	90.54 ± 0.89	68.08 ± 3.44	70.22 ± 3.41	N/A

The central observation is that the Top-1 standard deviations on the two datasets are nearly identical (2.94 pp on WLASL30 versus 2.89 pp on MS-ASL50). With $7\times$ fewer test samples on WLASL30, this is the strongest statement the experiment supports: *training stability is a property of the pipeline, not of a specific benchmark*. The pipeline does not need dataset-specific tuning to produce reproducible results on a second ASL corpus.

The absolute Top-1 figure on MS-ASL50 (70.20) is higher than on WLASL30 (58.82), but this is not evidence that MS-ASL is intrinsically easier. Three structural factors account for the gap. First, MS-ASL50 has approximately 6× more training samples (1,410 vs. 242), which is the dominant variable in this data regime. Second, the test set is 7× larger (236 vs. 34), so each individual misclassification moves Top-1 by approximately 0.42 pp instead of 2.94 pp; a finer measurement grid yields more representative values, which usually means closer to the true asymptotic performance. Third, MS-ASL clips are sourced from longer, better-lit YouTube educational material, which is reflected in the higher MediaPipe detection rate. The experiment therefore supports the claim that the pipeline reproduces; it does not support a claim about intrinsic dataset difficulty.

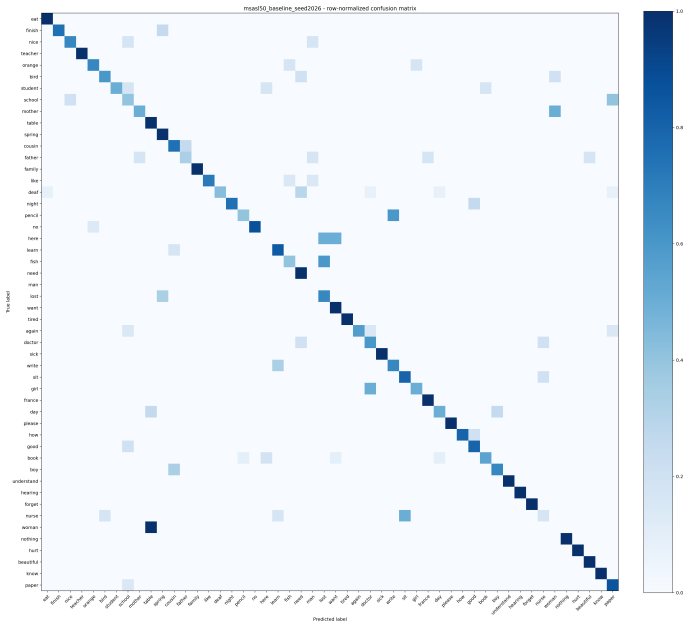


Figure 6.3: Row-normalised confusion matrix on the MS-ASL50 test split for seed 2026 (236 samples). The matrices for the other seeds show the same qualitative structure. Source: self-created from experiment outputs.

The confusion structure, as verified by the stored experiment outputs, is dominated by visually adjacent signs. The largest absolute confusion is *deaf* → *need* (4 of 14 samples, 28.6 %), followed by *pencil* → *write* and *fish* → *lost* (each 3 of 5, 60.0 %), *nurse* → *sit* (3 of 6, 50.0 %), and *mother* → *woman* (2 of 4, 50.0 %). These are recognisable as one-handed, location-similar pairs whose discriminative cue is fine-grained finger shape rather than gross trajectory.

The per-class metrics show three classes with 0.00 recall (*here*, *man*, *woman*, all with three test samples), and six further classes below 50 % recall (*need* at 0.25, *school* at 0.29, *nurse* at 0.33, *lost* at 0.33, *write* at 0.40, *doctor* at 0.43). At the other end, nineteen classes reach perfect recall on this split, including six with five or more support samples (*teacher* 9, *no* 7, *tired* 7, *deaf* 6, *please* 5, *like* 5). The headline Macro-F1 of 68.08 is therefore an average over a strongly bimodal per-class

distribution, which is consistent with the higher Macro-F1 standard deviation (3.44) compared to Top-1 (2.89): per-class recall is a finer-grained signal and amplifies seed-to-seed variation in which classes happen to be learned best.

6.7 Vocabulary Scaling

WLASL exposes progressively larger subsets of its 2,000-class vocabulary. Retraining the champion configuration on each subset quantifies how the system degrades as the recognition problem grows. The experiments in this section all use a single seed (42); multi-seed validation was not performed beyond 30 classes.

Table 6.6: Vocabulary scaling of the GRU+Attention champion on WLASL subsets. The 30-class row is the multi-seed mean from Table 6.4; all other rows are single-seed runs and standard deviations cannot be reported. *Train time* is the total wall-clock time of the training run on the NVIDIA RTX 3050 Ti Laptop GPU described in Section 6.1. Source: self-created from experiment manifests.

Vocabulary	Train / Val / Test	Top-1	Top-5	Best Epoch	Train time (s)
30	242 / 56 / 34	58.82 ± 2.94	78.43 ± 4.49	27–40	N/A
100	795 / 186 / 132	44.70	70.45	99	83
300	1,974 / 474 / 360	28.61	59.72	112	224
500	2,967 / 723 / 561	23.71	49.55	111	311
700	3,879 / 974 / 736	19.57	42.66	57	248
900	4,707 / 1,202 / 880	12.50	33.75	220	881
1,000	5,122 / 1,329 / 938	11.19	31.24	113	602
1,200	5,840 / 1,536 / 1,051	10.56	27.69	220	1,105
1,500	6,864 / 1,834 / 1,229	9.76	28.23	220	1,308

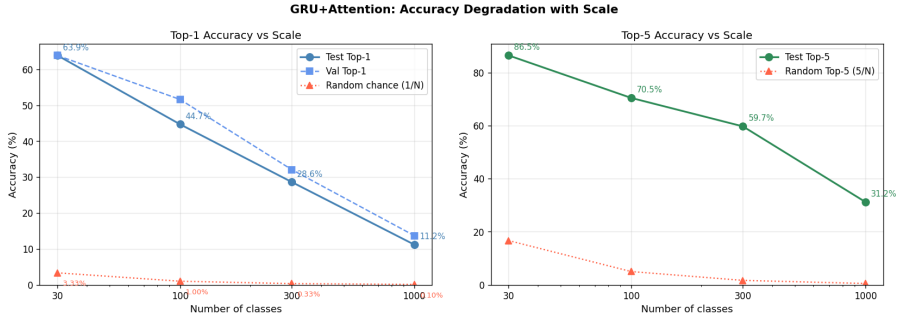


Figure 6.4: Top-1 accuracy as a function of vocabulary size on WLASL. Source: self-created from experiment outputs.

The relationship between vocabulary size and Top-1 accuracy is monotonically decreasing across the entire range and roughly linear in $\log(\text{num_classes})$ between 30 and 1,000 classes. The shape

of the curve has a clear interpretation: the model is not breaking. WLASL2000 has a median of approximately ten samples per class, which is below the regime where a discriminative neural sequence model can separate fine-grained gestural categories without external pretraining or stronger augmentation. The bottleneck is the per-class data budget, not the architecture.

A weaker secondary observation, restricted to the runs above 700 classes, is that the training curves of the larger subsets show greater sensitivity to early-stopping triggers; runs at 900, 1,200, and 1,500 classes saturate the maximum-epoch budget without an unambiguous improvement, while runs below 700 classes consistently early-stop earlier. This is the practical edge of the data-starvation regime: not a discrete failure point, but a regime where additional training time stops paying back.

The data-starvation interpretation is corroborated by the parallel-coordinates view in Figure 6.5, which projects every scaling run onto five normalised axes (mean videos per class, final train accuracy, validation Top-1, test Top-1, and test Top-5). The signature of the regime is the sharp drop between train accuracy and validation Top-1: at 30 classes the line stays high across all axes (mean 11.07 videos per class, train accuracy $99.00\,\%$, val Top-1 $63.85\,\%$), whereas at $1{,}500$ classes the same line collapses from $74.77\,\%$ train accuracy to $11.67\,\%$ validation Top-1 (4.58 videos per class, according to the scaling summary). The lines do not converge gradually, because the train-side stays comparatively flat while every validation- and test-side axis crashes. This is the textbook fingerprint of overfitting driven by sample scarcity rather than by an inadequate inductive bias.

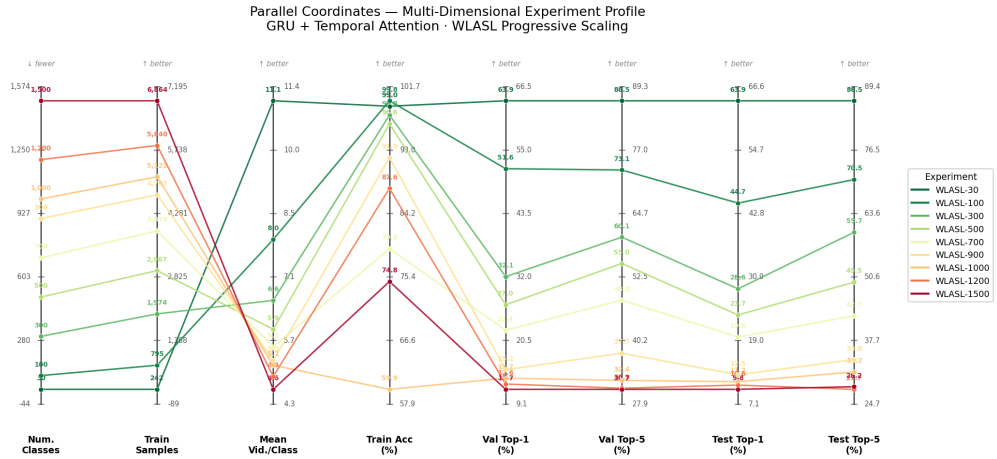


Figure 6.5: Parallel-coordinates view of the GRU+Attention scaling line on WLASL. Each line is one experiment, coloured from green (few classes) to red (many). The widening train–val gap is the hallmark of data starvation. Source: self-created from experiment outputs.

6.8 Limitations and Threats to Validity

The experiments above support several claims and undermine several others. The genuine limitations fall into six categories, addressed in turn.

The MediaPipe detection rate on WLASL is approximately $69\,\%$ at the frame level. Clips with motion blur, partially occluded hands, or hands moving outside the frame produce silent failures that the linear-interpolation step mitigates but does not erase. The interpolated frames carry no new

information, so any clip with sustained low detection rate effectively provides fewer informative frames than its nominal length suggests. The same caveat applies less forcefully to MS-ASL50, where the average detection rate is $83.9\,\%$.

The WLASL30 test split is small (34 samples). Even at three seeds, the standard deviation in Top-1 is bounded below by ≈2.94 pp because that is the resolution of one misclassification. The 58.82 ± 2.94 figure should be read as a range that any reasonable run will fall into, not as a precise estimate of the true generalisation accuracy. The 5-Fold CV mean of 63.85 partially mitigates this by averaging over five different test partitions, but the absolute resolution per fold is similar. This caveat is the reason MS-ASL50, with its 236-sample test split, is a more useful benchmark for fine-grained per-class analysis.

Beyond 30 classes, every reported result is single-seed. Differences between adjacent vocabulary sizes in Section 6.7 (e.g., 23.71 at 500 vs. 19.57 at 700) sit within a plausible seed-to-seed noise band, especially because the test sets at those sizes are large enough to make the per-sample resolution finer than at WLASL30. The downward trend itself is robust; the exact slope is not.

YouTube attrition on MS-ASL is $34.1\,\%$ of source clips. The MS-ASL50 subset reproducible from current YouTube state is therefore not identical to the one reproducible six months ago and will not be identical to the one reproducible six months from now. Any direct numerical comparison with results published against the original MS-ASL release [2] must be read with this caveat.

Only one architecture (GRU+Attention) was evaluated cross-dataset. A Transformer run on MS-ASL50 would directly test whether the Transformer's underperformance in Section 6.3 reflects the WLASL30 data regime or a deeper limitation of the architecture choice. That experiment is identified as a natural extension but was not executed within the scope of this thesis.

The augmentation pipeline was kept fixed throughout the chapter: rotation, scaling, translation, Gaussian noise, and horizontal flip. Horizontal flip in particular conflates left-handed and right-handed signers, which is acceptable for a small vocabulary one-hand pipeline but would need re-examination for two-hand signs and for downstream studies of signer-handedness bias. Class-weighted loss and minority-class oversampling were also not applied, despite the per-class disparities documented in Sections 6.5 and 6.6. Both are explicit future-work items.

Part II: Qualitative Evaluation

6.9 Qualitative Validation Protocol (In Progress)

The metrics reported in Part I measure recognition accuracy on fixed test sets under controlled landmark extraction. They do not, on their own, measure the usability of the mobile prototype when a real user tries to operate it. Closing that gap requires a structured protocol with human participants, the design of which is the subject of this section. The protocol is fixed; data collection and analysis are pending and will appear in a complementary report rather than in this thesis. The five subsections below cover the motivation and research questions, the participant population and recruitment criteria, the per-session task design, the instrumentation that records each session, and an analysis plan that defines how the qualitative session data will be reduced to quantitative metrics for that report.

6.9.1 Motivation and Research Questions

The qualitative protocol is necessary in addition to the quantitative experiments, not in place of them. Part I demonstrates that the model learns a recognisable representation of the MS-ASL50 vocabulary; the qualitative protocol is the first evidence that the surrounding system serves a user. Three properties of the prototype are invisible to Part I and motivate the protocol explicitly. The quantitative experiments consume clips that were segmented offline and processed by MediaPipe on the workstation, whereas the prototype consumes clips captured from a phone camera under uncontrolled lighting, framing, and execution speed. The Top-5 mechanism that this chapter relies on as an accountability fallback is only useful if participants actually consult it, and that behaviour is not observable from server-side metrics alone. Finally, the latency target stated as NFR-001 is meaningful only when measured end-to-end against real network and capture conditions, not against an isolated server benchmark.

The protocol is therefore designed to answer four research questions that Part I does not address. *RQ1:* Does the end-to-end latency of the prototype meet the $\leq$ 2-second target stated in NFR-001 under live use, as opposed to under isolated server benchmarking? *RQ2:* When the Top-1 prediction is incorrect, do participants consult the Top-5 fallback, or do they ignore it and treat the prediction as a single forced choice? *RQ3:* Which capture frictions, namely lighting, framing, distance from camera, and execution speed, dominate the user-perceived error rate, relative to the residual model error documented in Part I? *RQ4:* Does the Top-1 figure reported in Section 6.6 (70.20 % on MS-ASL50) overestimate or underestimate the accuracy a real user perceives in the field?

6.9.2 Participants and Recruitment

The target sample size is 10 to 25 participants. The lower bound is set so that variance across individuals does not dominate any aggregate observation; the upper bound is set so that the protocol stays within the regime of heuristic usability evaluation rather than crossing into a formal statistical study, which would require demographic stratification and controls that are out of scope here. The sample is not statistically representative of the deaf or hard-of-hearing population; it is an instrument for surfacing usability obstacles, not a census of the user base.

Inclusion criteria are an explicit interest in accessibility or assistive technology, or non-native ASL fluency motivated by an interest in learning the language. Native and fluent ASL signers are excluded from this round because they are not the population the prototype targets and because their natural execution speed and precision would bias the capture-friction observations toward zero. Recruitment is conducted through the author's personal and academic network, university accessibility-focused groups, and equivalent informal channels. Each participant signs a consent form acknowledging that the session captures server-side request logs, Likert responses, and free-text comments; that captured video frames are not persisted beyond the session; and that aggregated results may appear in the complementary report.

6.9.3 Task Design and Stimuli

Each session is structured around a fixed subset of $N = 10$ glosses drawn from the MS-ASL50 vocabulary. The choice of N is a compromise between three constraints: it must cover both ends of the bimodal per-class distribution documented in Section 6.6, it must fit a session of approximately 15 to 25 minutes without inducing fatigue, and it must include enough trials per participant for the per-trial outcome series to carry information. The 10 glosses are selected by stratification, not by

sampling: five are drawn from the classes that reached perfect recall on the seed-2026 test split (for example *teacher*, *no*, *tired*), and five are drawn from the classes whose recall fell below 0.50 (for example *here*, *man*, *woman*, *need*). The 1:1 stratification ensures that each session exposes both the best-case and the worst-case behaviour of the model.

The per-gloss procedure is the same for every participant. The participant first sees a short reference clip of a correctly executed sign, taken from a ground-truth video that was not part of the training set. The participant then performs the sign in front of the phone camera, and the prototype returns a Top-5 prediction with associated confidence scores. The participant marks three outcomes per trial: whether the Top-1 matches the target gloss, whether the Top-5 contains the target gloss, and an optional free-text comment. Each gloss is performed twice. The first attempt measures the user's learning curve with respect to the application itself; the second measures the steady-state accuracy after one round of feedback. The order of glosses is randomised per participant to neutralise ordering and fatigue effects.

6.9.4 Instrumentation: Server Logs and Google Form

Each session produces four streams of data. The first is server-side request data: every `POST /predict` call carries a timestamp, the measured end-to-end latency, the Top-K predictions with confidence scores, and an anonymised `participant_id`. The current FastAPI server documented in Section 5.1 exposes a `save_debug` hook that already persists the response payload to disk; for the qualitative sessions a thin request-logging middleware records the additional fields above into a structured JSONL file per session. The second stream is a post-session Google Form, organised in two blocks: a Likert block (1–5) covering perceived accuracy, frustration, intention to use the system in the future, and clarity of the Top-5 feedback, followed by a free-text block for capture-related obstacles. The third stream is the per-trial outcome that the participant marks during the session itself: the two binary fields *Top-1 matched the target* and *Top-5 contained the target*. These two fields, joined with the server logs, produce the per-trial time series on which the analysis plan in Section 6.9.5 operates. The fourth stream is observational: the author records, by hand and per gloss, retries and abandonment events. Detecting that a participant repeated a sign without success and moved on is a proxy for friction that the post-session Form does not capture as cleanly.

6.9.5 Analysis Plan: Quantifying the Qualitative

The reduction from session data to quantitative metrics is fixed in advance, in the spirit of preregistration, so that the qualitative chapter cannot be tuned to flatter the prototype after the fact. Six metric families will be derived once collection completes.

The first is the *user-perceived Top-1 accuracy*, defined as the number of trials whose Top-1 matches the target divided by the total number of trials. It is directly comparable to the 70.20 % reported in Section 6.6, with the caveat that trials are now over user-captured clips rather than over the MS-ASL50 test split. The second is the *user-perceived Top-5 accuracy*, defined analogously and comparable to the 90.54 %. The third is the *Top-5 utility rate*, defined as the proportion of Top-1-incorrect trials in which the participant reports having consulted the Top-5; this is the operational definition of RQ2 and is the only one that requires a self-report field. The fourth is the *median end-to-end latency* per trial, computed from the timestamps in the server log; the comparison against the 2-second NFR-001 target answers RQ1. The fifth is a *capture-friction frustration index*, defined as the per-category count of free-text comments classified by hand under the labels {*lighting*, *framing*, *distance*, *execution speed*, *other*}; the resulting distribution is the operational decomposition of RQ3. The sixth is

a per-item *Likert mean with* $95\,\%$ *bootstrap confidence interval* (10 000 resamples), used because at N between 10 and 25 the parametric normal-based interval is unreliable.

A single hypothesis test is pre-declared. To answer RQ4, the user-perceived Top-1 accuracy will be compared against the $70.20\,\%$ reference using a one-sample z-test of proportions; the conclusion will be reported as direction (*overestimates* or *underestimates*) and effect size, not as a binary significance verdict, given the modest sample size.

These analyses are pre-registered in the protocol and will be applied unchanged to the data once collected. The qualitative chapter does not anticipate any specific outcome.

Chapter 7

Ethical Implications and Social Impact

7.1 AI Based Systems Ethical Impact

An ethical analysis in Artificial Intelligence examines the implications, concerns and challenges associated with the development and deployment of these systems. As AI integrates into increasingly critical applications, technical evaluation must be paired with an explicit ethics review. This review rests on four general pillars: **Human Rights and Fairness** (preventing bias and discrimination), **Accountability** (clarifying who is responsible for the system's decisions), **Privacy and Security** (protecting user data and confidentiality), and **Transparency** (avoiding opaque black-box behaviour).

7.2 Ethical Evaluation Framework: Google's AI Principles

The framework adopted for this evaluation is **Google's AI Principles** [32], originally published in 2018 and refined annually; the primary reference for this analysis is the **2023 Progress Update** [33]. The framework was chosen because its seven principles are formulated as concrete objectives that map cleanly onto the design decisions of a research project, and because Google's annual progress reports demonstrate how the principles apply to systems that share characteristics with this work (computer vision and on-device inference). The seven principles are:

1. **Be socially beneficial:** address a real societal need and do so effectively.
2. **Avoid creating or reinforcing unfair bias:** ensure fair and inclusive treatment.
3. **Be built and tested for safety:** prioritise safety in design, validation, and operation.
4. **Be accountable to people:** provide transparency and user control.
5. **Incorporate privacy design principles:** protect user data by design.
6. **Uphold high standards of scientific excellence:** follow rigorous methodology.

7. **Be made available for uses that accord with these principles:** prevent misuse and align deployment with the framework.

The remainder of the chapter evaluates the project against each principle through quantitative metrics. Each metric is scored on a simplified {0, 5, 10} scale (not addressed, partially addressed, fully addressed). Some metrics derive from experimental results; others reflect design decisions or documentation practices. The summary table at the end of the chapter aggregates the metrics into per-principle averages and a global score.

7.3 Project Analysis Against Google's AI Principles

7.3.1 Principle 1: Be Socially Beneficial

Sign language is the primary means of communication for millions of deaf and hard-of-hearing individuals, yet a substantial communication barrier persists with the hearing population. An automatic ASL recognition system can act as a bridge in educational, medical, and everyday contexts.

- **M1.1: Real societal need.** The World Health Organization estimates over 430 million people worldwide have disabling hearing loss; the project targets a clear accessibility problem. *Score: 10.*
- **M1.2: Effectiveness of the solution.** The champion model achieves **63.85% Top-1 / 86.46% Top-5 (5-fold cross-validation, WLASL30)**, reproduced under the modular pipeline as 58.82% $\pm$ 2.94% Top-1 across three seeds, and extends to **70.20% $\pm$ 2.89% Top-1 / 90.54% $\pm$ 0.89% Top-5** on MS-ASL50. Accuracy degrades at scale (44.70% at 100 classes, 28.61% at 300, 11.19% at 1000), so the system is a research prototype rather than a deployment-ready translator. *Score: 5.*
- **M1.3: Accessibility of the technology.** Built on open-source tools (Python, PyTorch, MediaPipe) and public datasets (WLASL, MS-ASL), with the methodology documented in this thesis. The recognition pipeline was designed with mobile-first deployment in mind. *Score: 10.*

7.3.2 Principle 2: Avoid Creating or Reinforcing Unfair Bias

Sign language recognition is particularly bias-sensitive: signers vary in hand size, skin tone, signing speed, and style. Bias arises both at the perception layer (hand detection / landmark extraction operating on raw images) and at the recognition layer.

- **M2.1: Dataset diversity (word recognition).** WLASL aggregates videos from multiple online sources, providing implicit signer variation, but lacks demographic annotations (skin tone, age, hand morphology), so diversity cannot be quantified formally. *Score: 5.*
- **M2.2: Dataset diversity (hand detection and letter classification).** Hand detection used EgoHands ($\sim$4,800 egocentric images) and the Kaggle Hand Dataset ($\sim$4,200 third-person images). Letter classification used Sign Language MNIST (34,627 grayscale images) and the ASL Alphabet Dataset ($\sim$87,000 RGB images). None provide demographic metadata, so potential biases cannot be ruled out. *Score: 5.*
- **M2.3: Input representation fairness.** The word-recognition model consumes MediaPipe landmarks (21 keypoints $\times$ 3 coordinates) instead of raw pixels. Operating on geometric joint

positions rather than visual appearance reduces sensitivity to skin colour, lighting, and background, mitigating appearance-based bias at the recognition stage (though not at the upstream landmark extractor). *Score: 10.*

- **M2.4: Per-class performance analysis.** Per-class metrics were logged across all experiments, surfacing wide accuracy gaps (e.g. "what" 8.3%, "cool" 12.5%, vs. "orange" 100%, "table" 100%). The disparities are mostly explained by visual similarity between signs and small per-class sample counts rather than by demographic factors. Class-weighted loss or oversampling were not applied in this iteration but are facilitated by the existing per-class logging. *Score: 5.*

7.3.3 Principle 3: Be Built and Tested for Safety

In this context, safety means producing reliable predictions and avoiding misleading outputs that could cause misunderstandings in sensitive domains.

- **M3.1: Robust validation methodology.** Evaluation uses Stratified 5-Fold Cross-Validation (mean Top-1 **63.85% ± 4.50%**, mean Top-5 **86.46% ± 2.04%**), corroborated by a 3-seed multi-seed sweep on the same dataset (58.82% ± 2.94% Top-1) under the refactored pipeline. Test-Time Augmentation was deliberately avoided because it inflates reported accuracy without reflecting true generalisation. *Score: 10.*
- **M3.2: Scalability testing.** The model was retrained on progressively larger WLASL subsets (30 / 100 / 300 / 500 / 700 / 900 / 1000 / 1200 / 1500 classes), documenting accuracy degradation and an instability point above 700 classes. This stress-test establishes the boundaries within which the system can be considered reliable. *Score: 10.*
- **M3.3: Hand detection reliability.** The YOLOv8n hand detector reached approximately 98% mAP50 on the combined test set, validated across both source datasets. A reliable detector is a prerequisite for the recognition pipeline. *Score: 10.*
- **M3.4: Failure mode documentation.** Per-class confusion matrices identify systematically confused word pairs, but a confidence-threshold mechanism (so the system can abstain when uncertain) is not yet implemented and is needed before any production deployment. *Score: 5.*

7.3.4 Principle 4: Be Accountable to People

Accountability requires that the system is interpretable to its users and gives them meaningful control over its outputs.

- **M4.1: Model interpretability.** The Temporal Attention layer produces per-frame weights that can be inspected to understand which frames drove a prediction. This provides a degree of post-hoc interpretability that fully opaque architectures lack. *Score: 10.*
- **M4.2: Top-K predictions.** The system exposes Top-5 alongside Top-1. With Top-5 accuracy of 86.46% on WLASL30 (and 90.54% on MS-ASL50), the correct answer is almost always in the shortlist, allowing the user to make the final call rather than accepting a single forced prediction. *Score: 10.*
- **M4.3: Reproducibility.** All experiments are driven by versioned YAML configs and produce manifests with git commit, hyperparameters, and metrics. The main caveat is that some

WLASL videos rely on YouTube URLs that may decay over time; this is also an ethical concern about data provenance, since the original signers did not explicitly consent to use of their videos for ML research. *Score: 5.*

7.3.5 Principle 5: Incorporate Privacy Design Principles

Privacy applies to three distinct stages of the project: training the hand detector on real photographs, training the word recognition model on landmark abstractions, and serving inference from a mobile client.

- **M5.1: Data minimisation through landmarks (training).** Each video frame is reduced to 63 numerical values that discard the original visual content. Once landmarks are extracted, the original video is no longer needed and the processed data cannot reconstruct the signer's face, body, or background. *Score: 10.*
- **M5.2: Privacy in the hand detection phase.** Detector training relied on real photographs of identifiable individuals (EgoHands, Kaggle Hand). Although these datasets are publicly available for research, no independent privacy audit was conducted on their provenance, and the images were used as-is during training. *Score: 5.*
- **M5.3: Privacy in the inference pipeline.** The current FastAPI server receives full RGB frames (416×416) from the mobile app, processes them, and persists raw frames and cropped regions to disk without encryption or automatic expiration. This is the weakest privacy point. A natural mitigation is to move landmark extraction to the client device so that only the abstract 63-dimensional vector ever leaves the phone. *Score: 0.*
- **M5.4: K-anonymity considerations.** WLASL stores no identifying metadata about signers, and the landmark representation acts as a generalisation: different individuals performing the same sign yield similar feature vectors [34]. A formal k-anonymity algorithm was not applied, but the pipeline achieves a comparable effect for the recognition stage. The protection does not extend to the detector training data nor to the server inference path. *Score: 5.*

7.3.6 Principle 6: Uphold High Standards of Scientific Excellence

- **M6.1: Systematic model comparison (word recognition).** Multiple architectures were trained under matched conditions: CNN+LSTM (pixel-based), LSTM (landmark-based), plain GRU, GRU with enriched features, GRU + Temporal Attention, and a Transformer variant. *Score: 10.*
- **M6.2: Systematic model comparison (hand detection).** The hand detection stage compared YOLOv8n, SSDLite-MobileNetV3, and EfficientDet-D0 on the same combined dataset using standard detection metrics (mAP50, mAP50-95, latency). *Score: 10.*
- **M6.3: Statistical validation.** Stratified 5-Fold Cross-Validation reports mean and standard deviation across folds ($63.85\% \pm 4.50\%$ Top-1), and the modular pipeline contributes additional multi-seed runs on WLASL30 and MS-ASL50 to bound model-init variance. Pearson correlation analysis quantifies the relationship between vocabulary size and accuracy. *Score: 10.*
- **M6.4: Literature grounding.** The State of the Art chapter situates each architectural choice (CNN, RNN, LSTM, GRU, Attention, MediaPipe) in the established literature. *Score: 10.*

- **M6.5: Honest reporting of limitations.** The thesis explicitly documents accuracy degradation at scale, the ∼69% MediaPipe detection rate on WLASL, and the instability observed above 700 classes. Negative results are reported alongside positive findings. *Score: 10.*

7.3.7 Principle 7: Be Made Available for Uses That Accord with These Principles

- **M7.1: Intended use clarity.** The thesis frames the system as a research prototype for ASL word recognition, with assistive technology as the target use case. Scope and limitations are explicit. *Score: 10.*
- **M7.2: Misuse potential assessment.** Landmark vectors carry very limited misuse potential (no surveillance or facial recognition is recoverable from 63 floats per frame). The detector and the inference server, however, do process raw images and currently lack access controls and retention policies; this gap should be closed before any deployment beyond the research context. *Score: 5.*
- **M7.3: Open research contribution.** The methodology and code are documented in the thesis and in the repository. Trained models and processed datasets have not yet been published in a public artifact registry. *Score: 5.*

7.4 Impact Scale of Ethical Aspects

Table 7.1 consolidates all metrics, their individual scores, and the per-principle average.

Principle	ID	Metric	Score	Avg.
P1: Socially Beneficial	M1.1	Addresses a real societal need	10	**8.3**
	M1.2	Effectiveness of the solution	5	
	M1.3	Accessibility of the technology	10	
P2: Avoid Unfair Bias	M2.1	Dataset diversity (word recognition)	5	**6.3**
	M2.2	Dataset diversity (detection & letters)	5	
	M2.3	Input representation fairness	10	
	M2.4	Per-class performance analysis	5	
P3: Safety	M3.1	Robust validation methodology	10	**8.8**
	M3.2	Scalability testing	10	
	M3.3	Hand detection reliability	10	
	M3.4	Failure mode documentation	5	
P4: Account-ability	M4.1	Model interpretability	10	**8.3**
	M4.2	Top-K predictions	10	
	M4.3	Reproducibility	5	
P5: Privacy	M5.1	Data minimisation (landmark training)	10	**5.0**
	M5.2	Privacy in detection phase	5	
	M5.3	Privacy in inference pipeline	0	
	M5.4	K-anonymity considerations	5	
P6: Scientific Excellence	M6.1	Model comparison (word recognition)	10	**10.0**
	M6.2	Model comparison (hand detection)	10	
	M6.3	Statistical validation	10	
	M6.4	Literature grounding	10	
	M6.5	Honest reporting of limitations	10	
P7: Principled Availability	M7.1	Intended use clarity	10	**6.7**
	M7.2	Misuse potential assessment	5	
	M7.3	Open research contribution	5	
		Overall Average Score		**7.6**

Table 7.1: Ethical evaluation metrics based on Google's AI Principles [33]

7.5 Conclusions of the Ethical Analysis

The ethical evaluation yields an overall score of **7.6 out of 10**. The strongest pillars are scientific excellence (10.0/10) and safety (8.8/10); the weakest is privacy (5.0/10), driven entirely by the inference pipeline.

One architectural decision, the use of MediaPipe hand landmarks as input, propagates positive effects across multiple principles: it reduces appearance-based bias (P2), it implements data minimisation that protects identity by construction (P5), and it limits misuse potential because abstract landmarks cannot be repurposed for surveillance (P7). The same advantage, however, does not extend to the upstream stages that operate on raw images. The current FastAPI server receives, processes, and persists full RGB frames without encryption or automatic deletion, and the hand detector was trained on photographs of identifiable individuals. The most actionable mitigation is to relocate landmark extraction to the client device, so that only the 63-dimensional vector ever leaves the phone.

The intermediate scores in bias avoidance (6.3/10) and principled availability (6.7/10) reflect missing formal processes rather than poor design choices: no demographic audit of signers was conducted, the auxiliary detection datasets lack diversity metadata, and the server prototype has no access controls or retention policies. Honest reporting of these gaps, including the privacy weakness of the

inference pipeline identified in this chapter and the documented accuracy collapse at large vocabularies, is itself an ethical practice aligned with Principle 6 and is a prerequisite for any responsible deployment of the system.

Chapter 8

My Journey at UFV

Chapter 9

Conclusiones

9.1 General Conclusions

9.2 Conclusions regarding Project Objectives

9.3 Next Steps

Bibliography

[1] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2020, pp. 1459–1469.

[2] H. R. V. Joze and O. Koller, "Ms-asl: A large-scale data set and benchmark for understanding american sign language," in *British Machine Vision Conference (BMVC)*, 2019.

[3] S. Ramírez, *Fastapi*, https://fastapi.tiangolo.com/, Accessed: 2026, 2018.

[4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[5] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097–1105.

[6] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[7] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *European Conference on Computer Vision (ECCV)*, Springer, 2014, pp. 818–833.

[8] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," in *CVPR Workshop on Computer Vision for Augmented and Virtual Reality*, 2020.

[9] Y. Bengio, P. Simard, and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, vol. 5, no. 2, pp. 157–166, 1994. DOI: 10.1109/72.279181.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *NIPS 2014 Deep Learning Workshop*, 2014.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[13] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," *arXiv preprint arXiv:1409.0473*, 2014.

[14] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[15] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "Ssd: Single shot multibox detector," in *European Conference on Computer Vision (ECCV)*, Springer, 2016, pp. 21–37.

[16] M. Tan, R. Pang, and Q. V. Le, "Efficientdet: Scalable and efficient object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 781–10 790.

[17] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," pp. 779–788, 2016. DOI: 10.1109/CVPR.2016.91.

[18] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.

[19] G. Jocher, A. Chaurasia, and J. Qiu, *Ultralytics yolov8*, https://github.com/ultralytics/ultralytics, Accessed: 2025, 2023.

[20] O. Koller, "Quantitative survey of the state of the art in sign language recognition," *arXiv preprint arXiv:2008.09918*, 2020.

[21] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," vol. 164, Elsevier, 2021, p. 113 794.

[22] D. Bragg, O. Koller, M. Bellard, L. Berke, P. Boudreault, A. Braffort, N. Caselli, M. Huenerfauth, H. Kacorri, T. Verhoef, et al., "Sign language recognition, generation, and translation: An interdisciplinary perspective," pp. 16–31, 2019.

[23] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign language transformers: Joint end-to-end sign language recognition and translation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 10 023–10 033.

[24] K. Yin and J. Read, "Better sign language translation with STMC-transformer," in *Proceedings of the 28th International Conference on Computational Linguistics*, International Committee on Computational Linguistics, 2020, pp. 5975–5989. DOI: 10.18653/v1/2020.coling-main.525.

[25] M. Bohacek and M. Hruz, "Sign pose-based transformer for word-level sign language recognition," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)*, 2022, pp. 182–191. DOI: 10.1109/WACVW54805.2022.00024.

[26] J. Eunice, Y. Sei, and D. J. Hemanth, "Sign2pose: A pose-based approach for gloss prediction using a transformer model," *Sensors*, vol. 23, no. 5, p. 2853, 2023. DOI: 10.3390/s23052853.

[27] S. Albanie, G. Varol, L. Momeni, T. Afouras, J. S. Chung, N. Fox, and A. Zisserman, "Bsl-1k: Scaling up co-articulated sign language recognition using mouthing cues," in *European Conference on Computer Vision (ECCV)*, Springer, 2020, pp. 35–53. DOI: 10.1007/978-3-030-58621-8_3.

[28] S. Bambach, S. Lee, D. J. Crandall, and C. Yu, "Lending a hand: Detecting hands and recognizing activities in complex egocentric interactions," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2015, pp. 1949–1957.

[29] A. Nikkhah, *Hand dataset*, https://www.kaggle.com/datasets/armannikkhah/hand-dataset, Accessed: 2025, 2023.

[30] TechnicianLP, *Sign language mnist dataset*, https://www.kaggle.com/datamunge/sign-language-mnist, Accessed: 2025, 2017.

[31] Akash, *Asl alphabet dataset*, https://www.kaggle.com/grassknoted/asl-alphabet, Accessed: 2025, 2018.

[32] S. Pichai, *Ai at google: Our principles*, https://blog.google/technology/ai/ai-principles/, Published June 7, 2018. Accessed: 2026, 2018.

[33] Google, "Ai principles progress update 2023," Google, Tech. Rep., 2023, 5th edition of Google's annual AI Principles progress report. Accessed: 2026.

[34] L. Sweeney, "K-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, pp. 557–570, 2002. DOI: 10.1142/S0218488502001648.